Faster Bootstrapping for CKKS with Less Modulus Consumption *

Lianglin Yan[1,2[0009−0000−8378−9446]], Pengfei Zeng[1,2[0009−0007−5519−4868]], Heyang Cao[1,2[0009−0006−0180−0319]], Peizhe Song[1,2[0009−0005−1977−4828]], and Mingsheng Wang[1(✉)]

[1] State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS, Beijing, China
{caoheyang,songpeizhe,wangmingsheng}@iie.ac.cn
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
{yanlianglin20,zengpengfei20}@mails.ucas.ac.cn

Abstract. In fully homomorphic encryption, bootstrapping serves as a key component while also remaining the performance bottleneck of the scheme. Specifically, for CKKS bootstrapping, this bottleneck is reflected in significant computational overhead and modulus consumption.
In this work, we improve the CKKS bootstrapping with lower time complexity and less modulus consumption. We first propose a novel rescaling operation, called level-conserving rescaling, that acts on CoeffsToSlots for saving moduli. Secondly, we reconstruct the rotation keys and merge the plaintext-ciphertext multiplication and rescaling operations into the key-switching procedure, which reduces the time complexity of matrix-vector multiplication for matrices with $\leq$64 non-zero diagonals, albeit with increased space overhead. By combining the two methods in CoeffsToSlots in a non-trivial manner, we not only further accelerate the homomorphic linear transformations and save one level of moduli, but also reduce the total size of rotation keys.
Experiments demonstrate the practicability of our techniques. Compared to the state of the art, our approaches save one level of moduli, achieving a 20% $\sim$ 35% improvement in bootstrapping throughput and an 11.9% $\sim$ 15.2% reduction of rotation key size in CoeffsToSlots. Furthermore, with sufficient storage, our technology achieves up to 40% higher bootstrapping throughput than before, at the cost of doubling the rotation key size in CoeffsToSlots. The bootstrapping precision and failure probability remain identical to the previous method.

Keywords: Homomorphic encryption · CKKS bootstrapping · Linear transformation · Key-switching · Rescaling.

1 Introduction

Homomorphic encryption (HE) is an encryption scheme that enables one to compute on encrypted data without knowing secrets. The research in this area

has made great progress since Gentry's lattice-based construction [22] of the first fully homomorphic encryption (FHE) scheme in 2009. Due to the powerful ability to perform computations on encrypted data, FHE gradually becomes a popular solution for processing private information in an untrusted environment.

As a leveled HE (LHE) scheme relying on the Ring Learning with Errors (RLWE) problem [41], the Cheon–Kim–Kim–Song (CKKS) scheme is the only scheme supporting approximate homomorphic evaluation of algorithms over complex (or real) vectors, which is important in many applications like machine learning [35,36,40]. In recent years, CKKS has been improved in both algorithm and implementation [16,27,32,44,34,2] to achieve an efficient evaluation of complicated circuits (e.g., deep neural networks). Moreover, bootstrapping is required when evaluating the extremely deep circuits.

Bootstrapping is a technique that makes an LHE scheme become an FHE scheme. It takes a zero-level ciphertext as input and outputs the ciphertext of the same message at a high level. CKKS bootstrapping was first proposed by Cheon et al. [15]. Their core idea is raising the ciphertext level first and then eliminating the extra term in the plaintext via homomorphic modular reduction and linear transformations[3] (see Section 2.2 for details). The modular reduction was initially approximated by a sine function, and transformed into a Taylor polynomial for evaluation. To achieve a high performance, a series of subsequent works improved the evaluation method of the modular reduction function, such as Chebyshev interpolation [12,27], inverse sine function [37] and so on [7,28,38]. On the other hand, several works [33,12] focused on reducing the modulus consumption of bootstrapping, thus promoting the homomorphic evaluation ability of the bootstrapped scheme.

In addition, optimizing linear transformations also benefits bootstrapping. In CKKS bootstrapping, homomorphic linear transformations are essentially homomorphic matrix-vector multiplications [15,12,26], and the state-of-the-art linear transformations were introduced in [7]. The work in [7] adopted the matrix factorization algorithm [12,26], and introduced the double-hoisting BSGS algorithm for matrix-vector multiplications (detailed in Section 4.1). In this work, we aim to improve the performance of CKKS bootstrapping beyond the state of the art by optimizing the homomorphic linear transformations, especially the CoeffsToSlots procedure (introduced in Section 2.2).

1.1 Our Contributions

We introduce two new technologies, one of which saves the modulus level and the other accelerates the matrix-vector multiplications in CoeffsToSlots. They can be combined to achieve better results.

- We propose a novel *level-conserving rescaling* (LCR) technique that acts on CoeffsToSlots for saving moduli. It should be performed after ModRaise and before key-switching. (Section 3)

[3] In CKKS bootstrapping, homomorphic linear transformations represent the CoeffsToSlots and SlotsToCoeffs procedures.

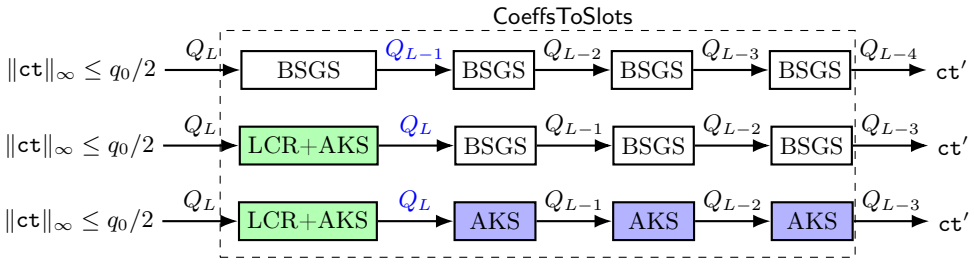

Fig. 1: Flowchart of the CoeffsToSlots procedure in prior works [7,8] (above), lossless strategy (middle) and time-memory trade-of strategy (below). As an example illustration, we assume that the CoeffsToSlots is factorized into 4 sparse diagonal matrices (i.e., 4-step decomposition). We omit the conjugation operations for simplicity.

- We present an innovative *aggregated key-switching* (AKS) technique, which speeds up the homomorphic matrix-vector multiplication when BSGS approach is discarded. Technically, we first observe that, in matrix-vector multiplication, the conventional Baby-Step Giant-Step (BSGS) approach can be removed without noticeable efficiency loss when the number of non-zero diagonals $r \le 32$. Then, we replace the BSGS approach with AKS method. The new key-switching operation substantially reduces the time complexity, and even compensates for the lost efficiency of discarding BSGS when $r = 64$. Complexity analysis shows that the improved matrix-vector multiplication algorithm is up to 2.5× (theoretical) faster than the previous BSGS method, while requiring more rotation keys in pre-computations. The idea of AKS may be of independent interest to the FHE community. (Section 4)
- We devise a new combination method for LCR and AKS by using the GHS-type key-switching[4] *non-trivially*. Applying to the first matrix-vector multiplication of CoeffsToSlots, LCR + AKS saves one level of moduli and reduces the time complexity. Notably, the use of GHS-type key-switching *further* decreases the time complexity and *reduces* the total size of the rotation keys, such that the combined algorithm is a *lossless improvement*. (Section 5)
- To demonstrate the practical improvements, we implement the proposed techniques in bootstrapping via two strategies: a lossless strategy and a time-memory trade-off strategy (illustrated in Fig. 1). The experimental results show that, compared to prior methods [7,8]: (1) Our methods save one level of moduli in bootstrapping and increase the remaining homomorphic capacity. (2) The lossless strategy achieves a 20%× ∼ 35%× throughput speedup and reduces the rotation key size in CoeffsToSlots by 11.9% to 15.2%. (3) The time-memory trade-off strategy achieves up to 40% speedup in bootstrapping

[4] GHS-type key-switching refers to the key-switching that employs an auxiliary modulus P to control the noise [23]. See Section 2.3.

throughput under similar parameters, and doubles the rotation key size in CoeffsToSlots, it would be useful when storage space is sufficient. (Section 6)

1.2 Technical Overview

We fist recap some necessary background concepts. Let $\mathcal{R}_Q = \mathbb{Z}[X]/(X^N+1, Q)$ and $\{Q_\ell = \prod_{i=0}^{\ell} q_i\}_{0\le\ell\le L}$ is the modulus chain for ciphertexts. To evaluate the decryption circuit homomorphically, standard CKKS bootstrapping executes four steps: ModRaise, CoeffsToSlots, EvalMod, and SlotsToCoeffs. The first operation, ModRaise, raises the ciphertext from the base level q_0 to the maximum level Q_L. Then, CoeffsToSlots packs the plaintext coefficients into slots. This can be done by homomorphically evaluating iDFT (i.e., a homomorphic matrix-vector multiplication). The subsequent EvalMod operation enables homomorphic modular reduction, and the final ciphertext is obtained following the SlotsToCoeffs operation, which retrieves messages from the slot side to the coefficient side.

This work mainly focuses on the ModRaise and CoeffsToSlots, especially the homomorphic matrix-vector multiplications. A widely used optimization for homomorphic matrix-vector multiplication is the Baby-Step Giant-Step (BSGS) approach. It divides the algorithm into two loops, each of which contains rotation and key-switching operations. Additionally, we also need to perform Rescale operation after each homomorphic multiplication. It multiples the ciphertext by a rescaling factor to reduce the ciphertext noise, consuming one level of moduli in the process.

The First Idea. Our first idea stems from an observation on the Rescale operation and the ModRaise procedure. We find that the ModRaise operation raises the modulus from q_0 to Q_L while preserving the ciphertext values, which implies that the output ciphertext $\mathtt{ct} \in \mathcal{R}_{Q_L}^2$ has coefficients bounded by $\|\mathtt{ct}\|_\infty \le q_0/2$. These coefficients are significantly smaller than the ring modulus Q_L. Therefore, we have $[\langle \mathtt{ct}, \mathtt{sk}\rangle]_{Q_L} = \langle \mathtt{ct}, \mathtt{sk}\rangle$ where $\mathtt{sk}$ is the secret key. After rescaling by q_L, $\mathtt{ct}$ becomes $\mathtt{ct}' = \lfloor \frac{1}{q_L} \cdot \mathtt{ct} \rceil$ and

$$[\langle \mathtt{ct}', \mathtt{sk}\rangle]_{Q_{L-1}} = \langle \mathtt{ct}', \mathtt{sk}\rangle = [\langle \mathtt{ct}', \mathtt{sk}\rangle]_{Q_L},$$

which means the modulus of $\mathtt{ct}'$ can be set to Q_L instead of Q_{L-1} without impacting the validity of the encryption. In other words, we can perform a *level-conserving* rescaling (LCR) operation on such a *small coefficient ciphertext* $\mathtt{ct}$ to preserve its level. However, using the level-conserving rescaling after ModRaise remains a challenge: rescaling is typically performed after the key-switching operation, as in CoeffsToSlots. Since the ciphertext coefficients after key-switching become uniformly distributed over $\mathbb{Z}_{Q_L}$, the level-conserving rescaling technique can no longer be applied.

An Attempt. To enable the LCR in CoeffsToSlots, we try to delay the key-switching and move the scalar multiplication and rescaling operations forward,

which undermines the structure of BSGS (see Section 3.2). As a result, this attempt preserves the modulus level but incurs unacceptable time and space overhead. To address this, we aim to reduce the algorithmic overhead in two steps: (1) The large overhead is because we break the BSGS approach. So, can we discard BSGS without causing significant efficiency losses? (2) We need to introduce additional techniques to enhance overall efficiency without BSGS.

Efforts in the Two Steps. Now, we temporarily forget about LCR and focus on the homomorphic matrix-vector multiplication algorithm. Let r be the number of non-zero diagonals of the matrix and r_1, r_2 be the number of loops in Baby-Step and Giant-Step, respectively. Then, it requires that $r = r_1 \cdot r_2$.

In step 1, we notice two facts: (1) The time complexity of the double-hoisting BSGS algorithm is minimized when the ratio r_1/r_2 is optimally set to 8 or 16 [7]. (2) Discarding BSGS approach in matrix-vector multiplication is equivalent to applying BSGS with $r_1 = r, r_2 = 1$ (i.e., $r_1/r_2 = r$). This implies that for $r \leq 16$, discarding BSGS inherently achieves the optimal ratio, incurring no efficiency loss theoretically. For $r = 32$, discarding BSGS results in only a slight loss of efficiency (30.4021 vs. 30.4018, see Table 1). In summary, we find that *for matrices with* $r \leq 32$, discarding BSGS incurs a small efficiency loss for double-hoisting matrix-vector multiplication.

In step 2, we introduce an *aggregated key-switching* (AKS) technique that accelerates the double-hoisting matrix-vector multiplication algorithm when the BSGS approach is not used. The main idea of AKS is merging the scalar multiplication and rescaling operations into key-switching procedure. Let s_1, s_2 be secret keys, P be the auxiliary modulus for key-switching and $\mathtt{ct} = (c_0, c_1) \in \mathcal{R}_{Q_\ell}^2$ be a ciphertext. We know that the switching key of key-switching from s_1 to s_2 is the ciphertext $\mathtt{evk}$, an encryption of Ps_1 with respect to secret s_2, and during key-switching, the term Ps_1 is multiplied by c_1. Based on this observation, we modify the switching key to the encryption of $\frac{Pms_1}{q_\ell}$, where m is a plaintext from DFT/iDFT matrix, and $1/q_\ell$ is the rescaling factor. Consequently, after key-switching (including ModDown), the term $\frac{ms_1}{q_\ell} \cdot c_1$ is generated, whereas the original key-switching produces $s_1 \cdot c_1$. This modification enables the scalar multiplication and rescaling on c_1 to be completed within the key-switching operation itself. As a result, only scalar multiplication and rescaling operations on c_0 need to be evaluated separately, reducing the overall time complexity.

Compared to the BSGS method, the improved matrix-vector multiplication algorithm with AKS *reduces the overall time complexity when* $r \leq 64$. This is because the efficiency loss from discarding BSGS is fully compensated by the aggregated key-switching approach when $32 \leq r \leq 64$ (see Table 2). We also note that this method introduces a time-space trade-off as it requires more rotation keys than the BSGS method.

The Combination. We now combine the LCR into the improved matrix-vector multiplication algorithm with the AKS approach. A key challenge arises because

the CRT composition step in the hybrid key-switching causes the rescaling operation to rely on the modulus $Q_{\ell-1}$ instead of Q_ℓ. This reliance critically obstructs the level preservation in key-switching (see Section 5.1). To resolve this, we employ the GHS-type key-switching while keeping the auxiliary modulus P unchanged—a non-trivial modification. This approach not only *preserves the modulus level*, but also *further lowers* the time complexity and *decreases* the total size of the rotation keys. Consequently, we obtain a *losslessly improved* matrix-vector multiplication algorithm for the first matrix in CoeffsToSlots with $r \leq 64$.

Up to now, we have three kinds of methods for matrix-vector multiplication algorithms in bootstrapping:

- The BSGS method, which is the state of the art prior to our work.
- The AKS method, which achieves a better efficiency than BSGS method when $r \leq 64$, at the cost of space overhead.
- The LCR+AKS method, which preserves the modulus level and outperforms the above two methods in both time and space overhead, but only applicable to the first matrix in CoeffsToSlots with $r \leq 64$ and only be used once.

Applying to Bootstrapping. In the state-of-the-art CKKS bootstrapping, the DFT/iDFT matrix is factorized into sparse diagonal matrices by utilizing the matrix factorization method from [26,12], typically with $r \approx 16$ or 32 (resp. 32 or 64) for 4-step (resp. 3-step) decomposition. Thus, our proposed algorithms (AKS and LCR+AKS) are applicable in real-world bootstrapping. We provide different bootstrapping strategies in application (refer to Fig. 1).

Lossless improvement (for limited storage). For the lossless strategy, since the LCR+AKS method outperforms BSGS in both time and space overhead, our improved bootstrapping saves one level of moduli, achieving a throughput $1.20 \sim 1.35$ times that of the previous method under similar parameters, and the rotation key size in CoeffsToSlots is $11.9\% \sim 15.2\%$ smaller than before (illustrated in Table 4).

Higher throughput with increased key materials (for sufficient storage). The LCR+AKS and AKS methods can be jointly applied to bootstrapping for better efficiency, at the cost of an increased rotation key size. In experiments, our time-memory trade-off strategy achieves up to 40% higher throughput while doubling the rotation key size in CoeffsToSlots, compared to prior bootstrapping methods [7,8]. We also propose more optimization techniques (including reusing AKS keys in circuit evaluation) to enhance the applicability of AKS method (refer to Appendix H).

1.3 Compatibility with Other Technologies

The only limitation of our technique is that we have not yet considered sparsely packed ciphertexts, as noted in Remark 1, and we believe this will be addressed in the near future. In fact, our method is compatible with most mainstream techniques for fully-packed ciphertext bootstrapping, including the sparse-secret

encapsulation [8] and the subring secret encapsulation [42]. We provide a detailed discussion in Section 6.3.

1.4 Related Works

According to the suggestion of Cheon et al. [15], it is essential to perform homomorphic Encode and Decode algorithms before and after EvalMod, by using CoeffsToSlots and SlotsToCoeffs, respectively. Homomorphic encoding/decoding involves homomorphic iDFT/DFT, which is a homomorphic matrix-vector multiplication, optimized by the Baby-Step Giant-Step (BSGS) approach in $\mathcal{O}(\sqrt{n})$ rotations and 1 depth. Later, Cheon et al. [12] and Han et al. [26] exploited the structure of DFT algorithm and factorized the DFT/iDFT matrix into several sparse block diagonal matrices to achieve a faster linear transform in $\mathcal{O}\left(\sqrt{\gamma}\log_\gamma n\right)$ rotations with $\mathcal{O}\left(\log_\gamma n\right)$ depth, where γ is a radix specified by users.

Several works improved the homomorphic rotations of matrix-vector multiplication. The hoisting technique, from Halevi and Shoup [25], preposes the Decomp step before the loop due to the property $[\phi_k(a)]_{q_i} = \phi_k([a]_{q_i})$ for $a \in \mathcal{R}_Q$ with $\phi_k : X \mapsto X^{5^k} \mod (X^N + 1)$, which reduces the number of Number Theoretic Transforms (NTTs) and CRT reconstructions. Bossuat et al. [7] optimized this hoisting technique by applying ϕ_k^{-1} to rotation keys and commuting Permute and MultSum steps. They also constructed a double-hoisting BSGS matrix-vector multiplication by delaying the ModDown step outside the loop, which significantly reduces the complexity of linear transformation. They analyzed the ratio of n_1/n_2 for the best performance. However, the high modulus consumption is still the weakness of bootstrapping. For a 25-level CKKS scheme, as shown in [7], only 10 levels are left after bootstrapping for continuing homomorphic computations.

To reduce the modulus consumption of bootstrapping, Kim et al. [33] replaced EvalMod by EvalRound, which is defined as EvalRound: $x \mapsto x - \mathsf{EvalMod}(x)$. For previous bootstrapping algorithms, smaller scaling factors for encoding iDFT matrices allow less modulus consumption in the rescaling of CoeffsToSlots, but yields a relative large noise. Such a noise would be further enlarged in subsequent procedures and finally impact the output precision. The subtraction operation in EvalRound gets rid of the relative large error, therefore allowing smaller scaling factors and saving moduli in CoeffsToSlots. The subsequent work EvalRound$^+$ [45] resolved some problems of EvalRound and further reduced the modulus consumption. Additionally, Cheon et al. proposed the Tuple-CKKS [14] for homomorphic multiplication with lower modulus consumption in the CKKS scheme. A recent work, OverModRaise [31], also aimed to save bootstrapping modulus by exploring CoeffsToSlots, and the ideas can be combined with our work.

1.5 Road-map

Section 2 introduces some preliminaries. In section 3, we show the level-conserving rescaling technique and present the initial attempt to use it. Section 4 contains the analysis of discarding BSGS and our improved matrix-vector multiplication algorithm with aggregated key-switching. We combine the proposed two approaches in section 5 and provide the evaluation results in section 6. Finally, we conclude our work in section 7.

2 Preliminaries

Notations. In this paper, we let N be a power-of-two integer, $q_0, q_1, \cdots, q_L, p_0, p_1, \cdots, p_{k-1}$ be $L+k+1$ distinct primes, and $Q_\ell = \prod_{i=0}^{\ell} q_i$, $P_j = \prod_{i=0}^{j} p_i$ for $0 \le \ell \le L$ and $0 \le j < k$. For simplicity, we write $Q = Q_L$ and $P = P_{k-1}$. We define $\mathcal{R}_Q = \mathbb{Z}_Q[X]/(X^N+1)$, the cyclotomic polynomial ring over the integers modulo Q. We also enforce $q_i \equiv 1 \pmod{2N}$ (as well as p_i) so that $\mathcal{R}_{PQ}$ is NTT-friendly. We write lower-case letters as integers or elements in $\mathcal{R}_Q$, while bold-faced lower-case letters represent (column) vectors, e.g., $\boldsymbol{v} = (v_0, v_1, \cdots, v_{m-1})^T$ where $v_i \in \mathbb{Z}$ or $\mathcal{R}_Q$ and $\boldsymbol{v}^T$ is the transpose vector of $\boldsymbol{v}$. Matrices over integers or polynomials are represented in bold-faced upper-case letters such as $\boldsymbol{M}$. The inner product of $\boldsymbol{a} = (a_0, a_1, \cdots, a_{n-1})$ and $\boldsymbol{b} = (b_0, b_1, \cdots, b_{n-1})$ is defined as $\langle \boldsymbol{a}, \boldsymbol{b} \rangle = \sum_{i=0}^{n-1} a_i \cdot b_i$. We denote $\|a\|_\infty$ the infinity norm of the coefficients vector of the polynomial a. $U(\cdot)$ denotes the uniform distribution over some set. Without causing ambiguity, we simplify $c_1 + c_2 \bmod (Q, X^N+1)$ (resp. $c_1 \cdot c_2 \bmod (Q, X^N+1)$) as $c_1 + c_2$ (resp. $c_1 \cdot c_2$) for $c_1, c_2 \in \mathcal{R}_Q$.

For reduction, $[x]_Q \in (-Q/2, Q/2]$ denotes the reduction of x modulo Q, and $\lfloor x \rfloor, \lceil x \rceil, \lfloor x \rceil$ represent rounding x to the floor, ceiling and nearest integer, respectively (if x is a polynomial, the operation is applied coefficient-wise).

For RNS-CKKS, we let $\mathcal{B} = \{q_0, q_1, \cdots, q_L\}$, $\mathcal{C} = \{p_0, p_1, \cdots, p_{k-1}\}$ and $\mathcal{D} = \mathcal{B} \cup \mathcal{C}$. Moreover, we assume that $L+1 = \alpha \cdot \texttt{dnum}$ and split the set $\mathcal{B}$ into $\texttt{dnum}$ segments, each with α numbers, i.e., $D_j = \prod_{i=j\alpha}^{(j+1)\alpha-1} q_i$ for $0 \le j < \texttt{dnum}$. For a polynomial $a \in \mathcal{R}_Q$, there are two RNS representations: one is the coefficient form, denoted as $[a]_{\mathcal{B}} = (a_0, \cdots, a_L) \in \mathcal{R}_{q_0} \times \cdots \times \mathcal{R}_{q_L}$, where $a_i = [a]_{q_i}$; the other one is NTT form, denoted as $\hat{a} = (\hat{a}_0, \cdots, \hat{a}_L) \in \mathbb{Z}_{q_0}^N \times \mathbb{Z}_{q_1}^N \times \cdots \times \mathbb{Z}_{q_L}^N$, where $\hat{a}_i = \texttt{NTT}(a_i)$. Unless otherwise noted, a denotes its representation in coefficient form and $\hat{a}$ denotes its representation in NTT form.

2.1 The Full-RNS CKKS Scheme

We review the CKKS scheme [17] and its homomorphic operations in a generalized perspective. The RNS representation of these operations are presented in Appendix A. Recall that RNS representation is just for better efficiency, rationale and correctness still depend on the original scheme.

Encoding and Decoding. Encoding and decoding algorithms are designed to perform transformation between *complex vector* and *plaintext polynomial* over $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$. The vector is called *message* with $N/2$ components, and each component corresponds to a *slot*. To describe the encoding/decoding procedures, we show the canonical embedding first.

Let $\zeta = e^{i\pi/N} \in \mathbb{C}$ be a primary $2N$-th root of unity, and define $\zeta_j = \zeta^{5^j}$ for $j = 0, 1, \cdots, N/2-1$. Let $\mathcal{P} = \mathbb{R}[X]/(X^N+1)$. A modified canonical embedding is defined as $\tau : \mathcal{P} \to \mathbb{C}^{N/2}, m(X) \mapsto \big(m(\zeta_0), m(\zeta_1), \cdots, m(\zeta_{N/2-1})\big)$. If we present $m(X)$ by its coefficient vector $(m_0, m_1, \cdots, m_{N-1})$, then this mapping can be rewritten as

$$\begin{bmatrix} 1 & \zeta_0 & \zeta_0^2 & \cdots \zeta_0^{N-1} \\ 1 & \zeta_1 & \zeta_1^2 & \cdots \zeta_1^{N-1} \\ \vdots & \vdots & \vdots & \ddots \quad \vdots \\ 1 & \zeta_{\frac{N}{2}-1} & \zeta_{\frac{N}{2}-1}^2 & \cdots \zeta_{\frac{N}{2}-1}^{N-1} \end{bmatrix} \begin{bmatrix} m_0 \\ m_1 \\ \vdots \\ m_{N-1} \end{bmatrix} = \begin{bmatrix} m(\zeta_0) \\ m(\zeta_1) \\ \vdots \\ m(\zeta_{N/2-1}) \end{bmatrix}, \tag{1}$$

which is essentially related to the Discrete Fourier Transformation (DFT). Similarly, the mapping $\tau^{-1} : \mathbb{C}^{N/2} \to \mathcal{P}$ involves an inverse DFT (iDFT). With DFT/iDFT, we can encode a complex vector $\boldsymbol{z} \in \mathbb{C}^{N/2}$ to a plaintext polynomial $\mathtt{pt} \in \mathcal{R}$ and decode $\mathtt{pt}$ back to $\boldsymbol{z}$.

- $\mathsf{Encode}(\boldsymbol{z}, \Delta)$. For message $\boldsymbol{z} \in \mathbb{C}^{N/2}$ and scaling factor Δ, output the plaintext polynomial
$$\mathtt{pt} = \lfloor \Delta \cdot \mathtt{iDFT}(\boldsymbol{z}; \bar{\boldsymbol{z}}) \rceil \in \mathcal{R},$$
where $\bar{\boldsymbol{z}}$ is the complex conjugation of $\boldsymbol{z}$ and $\lfloor \cdot \rceil$ discretizes elements from $\mathcal{P}$ to $\mathcal{R}$.
- $\mathsf{Decode}(\mathtt{pt}, \Delta)$. For plaintext polynomial $\mathtt{pt}$ and scaling factor Δ, compute
$$(\boldsymbol{z}; \bar{\boldsymbol{z}}) = \frac{1}{\Delta} \cdot \mathtt{DFT}(\mathtt{pt}),$$
and output the message $\boldsymbol{z} \in \mathbb{C}^{N/2}$.

Basic Operations. Let χ_{key} be the uniform distribution over the set of polynomials with coefficient vector in $\{0, \pm 1\}^N$ and Hamming weight h, χ_{enc} be the distribution over $\mathcal{R}$ with coefficients distributed over $\{-1, 0, 1\}$ with respective probabilities $\{1/4, 1/2, 1/4\}$, and χ_{err} be the distribution over $\mathcal{R}$ with coefficients following a Gaussian distribution with standard deviation σ. Let $\phi_k : m(X) \mapsto m(X^k) \pmod{(X^N + 1)}$ be the Galois mapping over $\mathcal{P}$. ℓ is an integer satisfying $0 \le \ell \le L$ and $\beta = \lceil (\ell + 1)/\alpha \rceil$.

- $\mathsf{CKKS.Setup}(1^\lambda)$: Given the security parameter λ, generate parameters N, small moduli $\{q_0, \cdots q_L\}$, $\{p_0, \cdots p_{k-1}\}$ and big moduli $Q, P, \{D_j\}_{0 \le j < \mathtt{dnum}}$, as required in notations. Generate Hamming weight h, Gaussian standard deviation σ to instantiate distributions χ_{key} and χ_{err}.

- $\mathsf{CKKS.SwitchKeyGen}(s_1, s_2)$: To generate the switching key for key-switching from s_1 to s_2, we sample $a_j \leftarrow U(\mathcal{R}_{PQ})$ and $e_j \leftarrow \chi_{\mathsf{err}}$, then return $\mathtt{evk}_{s_1 \to s_2} = (\mathtt{evk}_0, \mathtt{evk}_1, \cdots, \mathtt{evk}_{\mathtt{dnum}-1})$, where $\mathtt{evk}_j = \left(-a_j s_2 + P s_1 \cdot D_j^* \cdot \hat{D}_j + e_j, a_j\right) \in \mathcal{R}_{PQ}^2$ for $\hat{D}_j = Q/D_j$, $D_j^* = \left[\hat{D}_j^{-1}\right]_{D_j}$ and $0 \le j < \mathtt{dnum}$[5].
- $\mathsf{CKKS.KeyGen}(\cdot)$: This procedure generates the secret key $\mathtt{sk}$, the public key $\mathtt{pk}$, the relinearization key $\mathtt{rlk}$, the rotation key $\mathtt{rtk}$ and the complex conjugation key $\mathtt{conjk}$. Specifically, we sample $s \leftarrow \chi_{\mathsf{key}}$ and $\mathtt{sk} = (1, s)$. $\mathtt{pk} = (-as + e, a) \in \mathcal{R}_Q^2$ by sampling $a \leftarrow U(\mathcal{R}_Q)$ and $e \leftarrow \chi_{\mathsf{err}}$. $\mathtt{rlk} = \mathsf{CKKS.SwitchKeyGen}(s^2, s), \mathtt{rtk}_k = \mathsf{CKKS.SwitchKeyGen}(\phi_{5^k}(s), s)$, and $\mathtt{conjk} = \mathsf{CKKS.SwitchKeyGen}(\phi_{-1}(s), s)$.
- $\mathsf{CKKS.Enc}_{\mathtt{pk}}(\mathtt{pt})$: For $\mathtt{pt} \in \mathcal{R}$, sample $v \leftarrow \chi_{\mathsf{enc}}$ and $e_0, e_1 \leftarrow \chi_{\mathsf{err}}$, compute $\mathtt{ct} = v \cdot \mathtt{pk} + (\mathtt{pt} + e_0, e_1) \in \mathcal{R}_Q^2$.
- $\mathsf{CKKS.Dec}_{\mathtt{sk}}(\mathtt{ct})$: For $\mathtt{ct} = (c_0, c_1) \in \mathcal{R}_{Q_\ell}^2$, compute $\mathtt{pt} = [\langle \mathtt{ct}, \mathtt{sk} \rangle]_{Q_0} = [c_0 + c_1 s]_{Q_0} \in \mathcal{R}$.
- $\mathsf{CKKS.Add}(\mathtt{ct}, \mathtt{ct}')$: For $\mathtt{ct}, \mathtt{ct}' \in \mathcal{R}_{Q_\ell}^2$, compute $\mathtt{ct}_{\mathsf{Add}} = \mathtt{ct} + \mathtt{ct}' \in \mathcal{R}_{Q_\ell}^2$.
- $\mathsf{CKKS.KeySwitch}(d, \mathtt{evk})$: For $d \in \mathcal{R}_{Q_\ell}$ a polynomial, decompose d into $\boldsymbol{d} = \left([d]_{D_0}, [d]_{D_1}, \cdots, [d]_{D_{\beta-1}}\right) \in \mathcal{R}^\beta$, and return $(d_0', d_1') = \left\lfloor \frac{1}{P} \cdot \sum_{j=0}^{\beta-1} [d]_{D_j} \cdot \mathtt{evk}_j \right\rceil \in \mathcal{R}_{Q_\ell}^2$.
- $\mathsf{CKKS.Mult}(\mathtt{ct}, \mathtt{ct}', \mathtt{rlk})$: For $\mathtt{ct} = (c_0, c_1) \in \mathcal{R}_{Q_\ell}^2$, $\mathtt{ct}' = (c_0', c_1') \in \mathcal{R}_{Q_\ell}^2$, first compute $(d_0, d_1, d_2) = (c_0 c_0', c_0 c_1' + c_1 c_0', c_1 c_1') \in \mathcal{R}_{Q_\ell}^3$, then $\mathtt{ct}_{\mathsf{Mult}} = (d_0, d_1) + \mathsf{CKKS.KeySwitch}(d_2, \mathtt{rlk}) \in \mathcal{R}_{Q_\ell}^2$.
- $\mathsf{CKKS.Rescale}(\mathtt{ct})$: For $\mathtt{ct} \in \mathcal{R}_{Q_\ell}^2$, output $\mathtt{ct}' = \left\lfloor \frac{1}{q_\ell} \cdot \mathtt{ct} \right\rceil \in \mathcal{R}_{Q_{\ell-1}}^2$.
- $\mathsf{CKKS.Rotate}(\mathtt{ct}, k, \mathtt{rtk}_k)$: For $\mathtt{ct} = (c_0, c_1) \in \mathcal{R}_{Q_\ell}^2$, return $\mathtt{ct}_{\mathsf{rot}_k} = (\phi_{5^k}(c_0), 0) + \mathsf{CKKS.KeySwitch}(\phi_{5^k}(c_1), \mathtt{rtk}) \in \mathcal{R}_{Q_\ell}^2$.
- $\mathsf{CKKS.Conjugate}(\mathtt{ct}, \mathtt{conjk})$: For $\mathtt{ct} = (c_0, c_1) \in \mathcal{R}_{Q_\ell}^2$, return $\mathtt{ct}_{\mathsf{conj}} = (\phi_{-1}(c_0), 0) + \mathsf{CKKS.KeySwitch}(\phi_{-1}(c_1), \mathtt{conjk}) \in \mathcal{R}_{Q_\ell}^2$.

2.2 Overview of Bootstrapping Circuit

For LHE schemes, the level of ciphertexts decreases as the homomorphic multiplication is performed, thus the homomorphic operation must terminate after evaluating a certain depth circuit. In order to compute arbitrary deep circuits, bootstrapping is performed when the level is exhausted. Following Gentry's [22] blueprint, bootstrapping algorithm is essentially a "recryption", it evaluates the decryption circuit homomorphically to transfer the ciphertexts that no longer support homomorphic multiplication, to the relatively *fresher* ciphertexts for supporting further homomorphic computations. For a CKKS ciphertext, the level determines how fresh it is, therefore the first step of CKKS bootstrapping is raising modulus. However, raising a ciphertext $\mathtt{ct} \in \mathcal{R}_{q_0}$ to $\mathtt{ct}' \in \mathcal{R}_{Q_L}$ will introduce an extra term $q_0 I$ into the plaintext. Bootstrapping solves this problem by developing the homomorphic modular function.

[5] Here we use the hybrid key-switching method with RNS-decomposition and move the D_j^* to $\mathtt{evk}$ [24] for optimization.

Recall the CKKS scheme, a message (complex vector) becomes a plaintext (polynomial) after encoding, and a plaintext becomes a ciphertext (polynomial vector) after encrypting. A homomorphic operation over ciphertexts corresponds to a similar operation over messages (*slot side*) rather than the plaintexts (*coefficient side*), so the design of homomorphic circuits should be oriented to the operations required by messages over slot side. Unfortunately, the extra term $q_0 I$ appears on the coefficient side. Thus, we need to move the plaintext (coefficient side), including $q_0 I$, to the slot side before evaluating the modular function, and move back after that for recovery. These can be done by homomorphic encoding and homomorphic decoding.

Let $\mathtt{ct}$ be a ciphertext encrypting a plaintext $\mathtt{pt}$ (message $\boldsymbol{z}$ on slot side) at level 0, we can perform bootstrapping on $\mathtt{ct}$ as follows[6].

- $\mathsf{ModRaise}$: For $\mathtt{ct} \in \mathcal{R}_{q_0}^2$, it satisfies $[\langle \mathtt{ct}, \mathtt{sk} \rangle]_{q_0} = \mathtt{pt} + e$. We view it as $\mathtt{ct} \in \mathcal{R}_{Q_L}^2$ (In RNS, this step involves the fast basis conversion). One can see that the decryption becomes $[\langle \mathtt{ct}, sk \rangle]_{Q_L} = q_0 I + \mathtt{pt} + e$ for a polynomial $I(X)$ with integer coefficients.
- $\mathsf{CoeffsToSlots}$: From the perspective of the slot, we let $\boldsymbol{A}_{(k,j)} = \left(\zeta^{j \cdot 5^k}\right)_{0 \le k,j < N/2}$, then $\mathtt{iDFT}(\boldsymbol{z}; \bar{\boldsymbol{z}}) = \frac{1}{2}\left(\boldsymbol{A}^{-1}\boldsymbol{z} + \overline{\boldsymbol{A}^{-1}\boldsymbol{z}}; -i \cdot \boldsymbol{A}^{-1}\boldsymbol{z} + i \cdot \overline{\boldsymbol{A}^{-1}\boldsymbol{z}}\right) \in \mathbb{R}^N$ [7]. To evaluate encoding, we need two ciphertexts to store the homomorphic iDFT result since each ciphertext only provides $N/2$ slots. Homomorphically, compute $\mathtt{ct}' = \mathsf{MatMult}\left(\frac{1}{2}\boldsymbol{A}^{-1}, \mathtt{ct}\right)$ (see subsection 3.2), then $\mathtt{ct}_1 = \mathtt{ct}' + \mathsf{Conjugate}(\mathtt{ct}')$, $\mathtt{ct}_2 = -X^{N/2} \cdot \mathtt{ct}' + X^{N/2} \cdot \mathsf{Conjugate}(\mathtt{ct}')$. Note that the messages on the slot side of $\mathtt{ct}_1$ and $\mathtt{ct}_2$, denoted as $\boldsymbol{z}_1$ and $\boldsymbol{z}_2$, are the coefficients of $(q_0 I + \mathtt{pt} + e)/\Delta$ where Δ is the scaling factor[7].
- $\mathsf{EvalMod}$: Modular function cannot be evaluated directly in CKKS, thus we approximate it by $f(x) = \frac{q_0}{\Delta \cdot 2\pi} \sin\left(2\pi x \frac{\Delta}{q_0}\right)$ and use some polynomials to approximate this trigonometric function [7] in a fixed interval. Therefore, $\mathsf{EvalMod}$ can be done by evaluating these polynomials homomorphically. As a result, we have ciphertexts $\mathtt{ct}_3 = \mathsf{EvalMod}(\mathtt{ct}_1)$ and $\mathtt{ct}_4 = \mathsf{EvalMod}(\mathtt{ct}_2)$ with the messages $\boldsymbol{z}_3$ and $\boldsymbol{z}_4$ on slot side, respectively. The encrypting messages are the coefficients of $(\mathtt{pt} + e)/\Delta$.
- $\mathsf{SlotsToCoeffs}$: On the slot side, $\mathtt{DFT}(\mathtt{pt} + e) = \boldsymbol{A}\boldsymbol{z}_3 + i \cdot \boldsymbol{A}\boldsymbol{z}_4 = \boldsymbol{A}(\boldsymbol{z}_3 + i \cdot \boldsymbol{z}_4)$. For homomorphic evaluation, compute $\mathtt{ct}' = \mathtt{ct}_3 + X^{N/2} \cdot \mathtt{ct}_4$, then $\mathtt{ct}_{\mathsf{out}} = \mathsf{MatMult}(\boldsymbol{A}, \mathtt{ct}')$. Note that the message on the slot side has been recovered to the original $\boldsymbol{z}$ (approximately), thus $\mathtt{pt} + e$ is back to the coefficient side.

It is worth stating that $\mathtt{ct}_{\mathsf{out}} \in \mathcal{R}_{Q_\ell}^2$ for some $0 < \ell < L$ since all the homomorphic multiplications in bootstrapping need to consume levels.

[6] For the reasons in Remark 1, we don't consider the sparsely packed ciphertext and SubSum operation throughout this paper.

[7] The error term changes during the homomorphic computation, but we still use e to denote it for simplicity.

2.3 Key-Switching

There are three types of key-switching techniques so far. The first one is BV-type key-switching, proposed by Brakerski and Vaikuntanathan [11]. They use a basis $\boldsymbol{w}$ to decompose the ciphertext and encode the power-part of $\boldsymbol{w}$ into the switching keys. This method reduces the noise of key-switching but require more computations. The second one is GHS-type key-switching, proposed by Gentry, Halevi, and Smart in [23]. Their idea is to temporarily extend the size of modulus Q with another modulus P, then reduce modulus to control the noise. However, this lowers the multiplicative depth supported by the scheme. Another one is the hybrid method of BV-type and GHS-type key-switching techniques [23,29]. It is a trade-off between efficiency and the number of levels of the scheme. In this paper, we mainly focus on the hybrid method, and the GHS-type key-switching will be used in the algorithm from section 5.

Note that our proposed aggregated key-switching in section 4 differs from these three methods. While the aforementioned methods focus on noise control during key-switching, we redesign the switching keys to achieve higher efficiency. In practice, our approach is implemented in conjunction with one of the three methods mentioned above.

3 Level-Conserving Rescaling

In this section, we first propose a new rescaling technique and show the challenge of applying it to CoeffsToSlots. To overcome the challenge, we make an attempt. Although the result is not satisfactory, it is crucial for subsequent improvements in the later sections.

Let $\mathcal{P} = \mathbb{R}[X]/(X^N + 1)$. Throughout this paper, we sometimes use non-integer polynomials (in $\mathcal{P}$) as plaintexts for the sake of analysis, as in [17].

3.1 The Rationales

Existing Rescaling. So far, for leveled homomorphic encryption schemes, the only way to reduce the error of a ciphertext is performing modulus switching, which is called *rescaling* in CKKS. While reducing errors, the ciphertext modulus should be reduced accordingly to maintain the validity of the ciphertext.

Specifically, for a ciphertext $\mathtt{ct} \in \mathcal{R}^2_{Q_\ell}$ encrypting plaintext $\mathtt{pt}$ with respect to secret key $\mathtt{sk}$, it is well-known that $[\langle \mathtt{ct}, \mathtt{sk} \rangle]_{Q_\ell} = \mathtt{pt} + e$ where e is the error and $\|\mathtt{pt} + e\|_\infty < Q_\ell/2$. We reformulate it as $\langle \mathtt{ct}, \mathtt{sk} \rangle = \mathtt{pt} + e + kQ_\ell$ for a polynomial k with integer coefficients. After rescaling by q_ℓ, we have ciphertext $\mathtt{ct}' = \left\lfloor \frac{1}{q_\ell} \cdot \mathtt{ct} \right\rceil$ that satisfies $\langle \mathtt{ct}', \mathtt{sk} \rangle \approx (\mathtt{pt} + e)/q_\ell + kQ_{\ell-1}$. To eliminate the term $kQ_{\ell-1}$, the modulus of $\mathtt{ct}'$ should be $Q_{\ell-1}$ rather than Q_ℓ. Moreover, if $\|(\mathtt{pt} + e)/q_\ell\|_\infty < Q_0/2$, Lemma 1 states that the modulus of $\mathtt{ct}'$ can be any element of $\{Q_i\}_{0 \le i \le \ell-1}$ (via the application of DropLevel), but cannot be Q_ℓ or any other larger modulus.

Lemma 1. *Let* $\{Q_i = \prod_{j=0}^{i} q_j\}_{0 \le i \le L}$ *be the moduli of CKKS. For any ciphertext* $\mathtt{ct} \in \mathcal{R}_{Q_\ell}^2$ *encrypting* $\mathtt{pt}$ *with error* e*, it can be (approximately) decrypted to* $\mathtt{pt}$ *with any modulus* Q_i *if* $\|\mathtt{pt} + e\|_\infty < Q_i/2$ *and* $Q_i | Q_\ell$*.*

Proof. Let $\mathtt{ct}$ encrypt the plaintext $\mathtt{pt}$ with respect to the secret key $\mathtt{sk}$, we know that $[\langle \mathtt{ct}, \mathtt{sk} \rangle]_{Q_\ell} = \mathtt{pt} + e$ where e is the error term. Then $\langle \mathtt{ct}, \mathtt{sk} \rangle = \mathtt{pt} + e + kQ_\ell$ for k a polynomial with integer coefficients, it is obvious that for any $Q_i | Q_\ell$ and $\|\mathtt{pt} + e\|_\infty < Q_i/2$, $[\langle \mathtt{ct}, \mathtt{sk} \rangle]_{Q_i} = \mathtt{pt} + e$ holds. □

To save modulus consumption of homomorphic operation, we would like to know whether there exists a rescaling operation that does not reduce modulus while maintaining the correct plaintext. That is, under what conditions can Q_ℓ be the modulus of the rescaled ciphertext $\mathtt{ct}'$? The above analysis implies that $\|\langle \mathtt{ct}, \mathtt{sk} \rangle\|_\infty < Q_\ell/2$ seems like an answer, but it is almost impossible since the coefficients of $\mathtt{ct}$ are usually uniform over $\mathbb{Z}_{Q_\ell}$ due to the RLWE assumption.

Our Rescaling. Fortunately, an in-depth observation into ModRaise makes *rescaling without modulus consumption* possible. Recall the process of ModRaise, the output ciphertext $\mathtt{ct}$ is at level L, but is not uniformly distributed over $\mathcal{R}_{Q_L}^2$. More specifically, it comes from the same $\mathtt{ct} \in \mathcal{R}_{q_0}^2$ and satisfies that $\langle \mathtt{ct}, \mathtt{sk} \rangle = q_0 I + \mathtt{pt} + e \ll Q_L/2$. After rescaling by q_L, $\mathtt{ct}$ becomes $\mathtt{ct}'$ and $[\langle \mathtt{ct}', \mathtt{sk} \rangle]_{Q_{L-1}} = [\langle \mathtt{ct}', \mathtt{sk} \rangle]_{Q_L} \approx (q_0 I + \mathtt{pt} + e)/q_L$, which means that the modulus of $\mathtt{ct}'$ stays at Q_L after rescaling. We call this new rescaling operation *level-conserving rescaling* (LCR).

Next, we show the concrete algorithm of level-conserving rescaling, and propose Theorem 1 that provides the correctness and the conditions for applying this new rescaling operation.

The algorithm of level-conserving rescaling is the same as CKKS.Rescale, except that the output $\mathtt{ct}'$ is treated as a ciphertext in level ℓ. Alternatively, it can also be viewed as a combination of an ordinary rescaling from Q_ℓ to $Q_{\ell-1}$ and a modraise from $Q_{\ell-1}$ to Q_ℓ. The operation is presented as follow (see Appendix B.1 for the RNS version).

- $\mathsf{LCRescale}(\mathtt{ct})$: For $\mathtt{ct} \in \mathcal{R}_{Q_\ell}^2$, output $\mathtt{ct}' = \lfloor \frac{1}{q_\ell} \cdot \mathtt{ct} \rceil \in \mathcal{R}_{Q_\ell}^2$.

Theorem 1. *For ciphertext* $\mathtt{ct}$ *regarding to plaintext* $\mathtt{pt}$*, secret key* $\mathtt{sk}$ *and modulus* Q_ℓ*, if* $\|\langle \mathtt{ct}, \mathtt{sk} \rangle\|_\infty < Q_\ell/2$*, then* $\mathtt{ct}' = \mathsf{LCRescale}(\mathtt{ct})$ *is the ciphertext of* $\mathtt{pt}/q_\ell$ *with respect to the same secret* $\mathtt{sk}$ *and modulus* Q_ℓ*.*

Proof. According to the decryption, we know $\langle \mathtt{ct}, \mathtt{sk} \rangle = \mathtt{pt} + e + kQ_\ell$ with $\|\mathtt{pt} + e\|_\infty < Q_\ell/2$ for an error e and a polynomial k with integer coefficients. If $\|\langle \mathtt{ct}, \mathtt{sk} \rangle\|_\infty < Q_\ell/2$, then $k = 0$, i.e., $\langle \mathtt{ct}, \mathtt{sk} \rangle = \mathtt{pt} + e$. After level-conserving rescaling, $\mathtt{ct}' = \left\lfloor \frac{1}{q_\ell} \cdot \mathtt{ct} \right\rceil = \frac{1}{q_\ell} \cdot \mathtt{ct} + \boldsymbol{r}$ where $\boldsymbol{r}$ is the rounding term with $\|\boldsymbol{r}\|_\infty \le 1/2$. We have

$$[\langle \mathtt{ct}', \mathtt{sk} \rangle]_{Q_\ell} = \left[\frac{1}{q_\ell} \cdot \langle \mathtt{ct}, \mathtt{sk} \rangle + \langle \boldsymbol{r}, \mathtt{sk} \rangle \right]_{Q_\ell} = \frac{\mathtt{pt}}{q_\ell} + e'$$

where $e' = \frac{e}{q_\ell} + \langle \boldsymbol{r}, \mathtt{sk} \rangle$ and $\|\langle \boldsymbol{r}, \mathtt{sk} \rangle\|_\infty \le \frac{h+1}{2}$. □

The Challenge. Theorem 1 claims that such level-conserving rescaling can only be applied to those ciphertexts that have relatively small coefficients, i.e., $\|\langle \mathtt{ct}, \mathtt{sk}\rangle\|_\infty < Q_\ell/2$. We also find that the ciphertext after ModRaise exactly satisfies this requirement. However, rescaling is usually performed after homomorphic multiplication, and homomorphic multiplication requires the key-switching operation, which involves an inner-product between the ciphertext and $\mathtt{evk}$. This product would make the coefficients of the ciphertext uniform over $\mathbb{Z}_{Q_\ell}$, and then level-conserving rescaling becomes useless. Therefore, the application of the level-conserving rescaling technique is still significantly challenging. In the next subsection, we try to apply it to CoeffsToSlots.

Remark 1. In sparsely packed ciphertexts, the SubSum algorithm is performed after ModRaise [15,7], which contains the rotations and key-switching operations. As mentioned before, key-switching would break the small coefficient property of the ciphertexts and make level-conserving rescaling useless. Therefore, in this paper, we assume that all ciphertexts are fully packed.

3.2 An Attempt: level-conserving Matrix×Vector

As the procedure immediately following ModRaise, CoeffsToSlots should be explored for applying the level-conserving rescaling. In CoeffsToSlots, rescaling is performed at the end of the homomorphic matrix-vector multiplication. So we try to modify the matrix-vector multiplication algorithm by moving rescaling forward and see what will happen.

Original Algorithm. In CKKS bootstrapping [15], homomorphic matrix-vector multiplication involves plaintext-ciphertext multiplications and rotations. More formally, let $n = N/2$, for a matrix $\boldsymbol{M} = (M_{i,j})_{0\le i,j<n} \in \mathbb{C}^{n\times n}$ and a ciphertext $\mathtt{ct}$ with message $\boldsymbol{z} \in \mathbb{C}^n$ on slots, the goal of linear transformation is to compute a ciphertext $\mathtt{ct}'$ with message $\boldsymbol{Mz}$ on slots. Let $\rho_j(\cdot)$ denote the cyclic shift of j positions to the left ($j < 0$ means cyclic shift to right), $\boldsymbol{u}_j = (M_{0,j}, M_{1,j+1}, \cdots, M_{n-j-1,n-1}, M_{n-j,0}, \cdots, M_{n-1,j-1}) \in \mathbb{C}^n$ denote the shifted diagonal vector of $\boldsymbol{M}$ for $0 \le j < n$, as presented in [25,15], then the matrix-vector multiplication can be represented as

$$\boldsymbol{M}\cdot\boldsymbol{z} = \sum_{j=0}^{n-1}(\boldsymbol{u}_j \odot \rho_j(\boldsymbol{z})) \tag{2}$$

where $\odot$ is the Hadamard product of vectors. Using the idea of Baby-Step Giant-Step (BSGS) algorithm, it can be optimized to

$$\boldsymbol{M}\cdot\boldsymbol{z} = \sum_{j=0}^{n_2-1}\rho_{n_1\cdot j}\left(\sum_{i=0}^{n_1-1}\rho_{-n_1\cdot j}(\boldsymbol{u}_{n_1\cdot j+i})\odot\rho_i(\boldsymbol{z})\right) \tag{3}$$

where $n_1 = \mathcal{O}(\sqrt{n})$ and $n_2 = n/n_1$ are both integers. Homomorphically, the matrix-vector multiplication is described as follows, which requires $\mathcal{O}(n_1 + n_2)$

rotations and $\mathcal{O}(n)$ scalar multiplications (plaintext-ciphertext multiplications).

$\mathsf{MatMult}(\boldsymbol{M}, \mathtt{ct})$: BSGS algorithm of matrix-vector multiplication in [15].
Input: A matrix $\boldsymbol{M} \in \mathbb{C}^{n\times n}$, a ciphertext $\mathtt{ct} \in \mathcal{R}^2_{Q_\ell}$, $n = n_1 n_2$, a scaling factor Δ and the required rotation keys $\{\mathtt{rtk}_k\}_{0\le k<n}$.
Output: A ciphertext $\mathtt{ct}' \in \mathcal{R}^2_{Q_{\ell-1}}$.
Step 1 (Precomputation): $\mathtt{pt}_{n_1\cdot j+i} = \mathsf{Encode}(\rho_{-n_1\cdot j}(\boldsymbol{u}_{n_1\cdot j+i}), \Delta) \in \mathcal{R}$ for $0 \le i < n_1, 0 \le j < n_2$.
Step 2 (Baby-Step): Compute $\mathtt{ct}_j = \sum_{i=0}^{n_1-1}\big(\mathtt{pt}_{n_1\cdot j+i} \cdot \mathsf{CKKS.Rotate}(\mathtt{ct}, i, \mathtt{rtk}_i)\big) \in \mathcal{R}^2_{Q_\ell}$ for $0 \le j < n_2$.
Step 3 (Giant-Step): Compute $\mathtt{ct}^* = \sum_{j=0}^{n_2-1} \mathsf{CKKS.Rotate}(\mathtt{ct}_j, n_1\cdot j, \mathtt{rtk}_{n_1\cdot j})$.
Step 4 (Rescaling): Compute $\mathtt{ct}' = \mathsf{CKKS.Rescale}(\mathtt{ct}^*) \in \mathcal{R}^2_{Q_{\ell-1}}$.

Tentative Modification. In the BSGS algorithm of matrix-vector multiplication, the rescaling operation is after rotations, and the key-switching operation in rotations breaks the small coefficient property of ciphertext. Therefore, to use the level-conserving rescaling, we need to discard the BSGS algorithm and delay the rotations.

We rewrite Eq.(2) as

$$\boldsymbol{M}\cdot\boldsymbol{z} = \sum_{j=0}^{n-1} \rho_j(\rho_{-j}(\boldsymbol{u}_j)\odot\boldsymbol{z}), \tag{4}$$

and put the rescaling before homomorphic rotations. Moreover, we replace the rescaling operation by the level-conserving rescaling, to obtain a *level-conserving matrix-vector multiplication* that does not consume any modulus. The correctness of the proposed algorithm relies on the Theorem 1 that has been proven.

level-conserving matrix-vector multiplication algorithm.
Input: A matrix $\boldsymbol{M} \in \mathbb{C}^{n\times n}$ with $\{\boldsymbol{u}_j\}_{0\le j<n}$ its shifted diagonal vectors, ciphertext $\mathtt{ct} \in \mathcal{R}^2_{Q_L}$ is the output of $\mathsf{ModRaise}$, a scaling factor Δ and the required rotation keys $\{\mathtt{rtk}_j\}_{0\le j<n}$.
Output: A ciphertext $\mathtt{ct}' \in \mathcal{R}^2_{Q_L}$.
Step 1 (Precomputation): $\mathtt{pt}_j = \mathsf{Encode}(\rho_{-j}(\boldsymbol{u}_j), \Delta) \in \mathcal{R}$ for $0 \le j < n$.
Step 2 (Multiplication and rescaling): Compute $\mathtt{ct}_j = \mathsf{LCRescale}(\mathtt{pt}_j \cdot \mathtt{ct}) \in \mathcal{R}^2_{Q_L}$ for $0 \le j < n$.
Step 3 (Rotations): Compute $\mathtt{ct}' = \sum_{j=0}^{n-1} \mathsf{CKKS.Rotate}(\mathtt{ct}_j, j, \mathtt{rtk}_j) \in \mathcal{R}^2_{Q_L}$.

Results of the Attempt. It is easy to see that the level-conserving matrix-vector multiplication consumes no modulus, thus saving one level of moduli in bootstrapping. However, without the BSGS algorithm, the algorithmic overhead (both time and space) is worse than before. The modified algorithm requires $n-1$ rotations while the original method requires $n_1+n_2-2 \approx \mathcal{O}(\sqrt{n})$ rotations, which also means more memory is needed to store the rotation keys. The modified algorithm also repeats rescaling operation n times while the BSGS

method only needs 1 rescaling. Overall, this result is not satisfactory. Despite saving the modulus, it seems that the overhead loss is not worth it.

Next Directions. We believe that the modification in the attempt for saving modulus is on the right path, but we need to reduce the algorithmic overhead. A high-level analysis does not seem to provide solutions, we will conduct a deeper analysis of the matrix-vector multiplication algorithm. Note that the complexity increased because we discard the idea of BSGS. So we consider, is there a case where discarding BSGS does not bring efficiency loss? (We find the case in subsection 4.1). Additionally, We can introduce more techniques to further reduce the complexity, which is exactly what we will do in subsection 4.2.

4 Improved Matrix-Vector Multiplication with Aggregated Key-Switching

In this section, we temporarily forget about the level-conserving rescaling proposed in Section 3 and focus on improving matrix-vector multiplication without BSGS. In subsection 4.1, we recall the existing double-hoisting BSGS matrix-vector multiplication in [7] and provide a theoretical complexity analysis. We notice that when using the matrix factorization approach [26,12] in linear transformations of bootstrapping, the number of non-zero diagonals of the factorized matrices is small. In this case, discarding BSGS seems to incur no (or very small) efficiency loss, especially with non-zero diagonals ≤ 32. Furthermore, in subsection 4.2, we introduce the aggregated key-switching approach to further reduce the computational overhead of matrix-vector multiplication without BSGS for matrices with non-zero diagonals ≤ 64.

For a detailed complexity analysis, we consider the full-RNS CKKS in this section. The basic homomorphic operations of the RNS-CKKS are presented in Appendix A. Unless otherwise stated, notations in this section are consistent with those in section 2 and Appendix A. In RNS representation, we assume that all polynomials are presented in the NTT form by default, and would be converted into the coefficient form when required. Additionally, let $n = N/2$, $n_1 n_2 = n$, $\mathcal{B}_\ell = \{q_0, q_1, \cdots, q_\ell\}$, $\mathcal{D}_\ell = \mathcal{B}_\ell \cup \mathcal{C}$ for $0 \leq \ell \leq L$, and the number of separated segments $\beta = \lceil (\ell+1)/\alpha \rceil$. Let $\hat{D}_j = Q/D_j$, and $D_j^* = \left[\hat{D}_j^{-1}\right]_{D_j}$ denoting the inverse of $\hat{D}_j$ modulo D_j for $0 \leq j < \beta$.

4.1 Existing Double-Hoisting BSGS Matrix-Vector Multiplication

We discuss the double-hoisting BSGS matrix-vector multiplication algorithm in [7] since it contains most of the existing optimizations related to linear transformations, such as the hybrid key-switching approach, the improved key-switching keys (including both $\hat{D}_j$ and D_j^*), the optimized hoisting-rotations and the

double-hoisting BSGS algorithm by delaying the ModDown step. For simplicity, we let φ_k denote the automorphism ϕ_{5^k} (defined in subsection 2.1). In precomputations, we need to compute the modified rotation keys

$$\widetilde{\mathtt{rtk}}_{k,j} = \left(\widetilde{\mathtt{rtk}}^0_{k,j}, \widetilde{\mathtt{rtk}}^1_{k,j}\right) = \left(\left[-a_j\varphi_k^{-1}(s) + Ps\cdot\hat{D}_j\cdot D_j^* + e_j\right]_{PQ_L}, [a_j]_{PQ_L}\right)$$

where $0 \le j < \mathtt{dnum}$ and k is the rotation position number. For matrix $\boldsymbol{M} \in \mathbb{C}^{n\times n}$, we encode its shifted diagonal vector into

$$\mathtt{pt}_{n_1\cdot t+i} = \mathsf{Encode}(\rho_{-n_1\cdot t}(\boldsymbol{u}_{n_1\cdot t+i}), \Delta) \in \mathcal{R}$$

for $0 \le i < n_1, 0 \le t < n_2$, according to Eq.(3).

The double-hoisting BSGS algorithm of matrix-vector multiplication in [7].

Input: Ciphertext $\hat{\mathtt{ct}} = (\hat{\boldsymbol{c}}_0, \hat{\boldsymbol{c}}_1) \in (\prod_{j'=0}^{\ell}\mathbb{Z}_{q_{j'}}^N)^2$, plaintexts $\{\hat{\mathtt{pt}}_{n_1\cdot t+i}\}$ and the rotation keys $\{\widehat{\mathtt{rtk}}_{i,j}, \widehat{\mathtt{rtk}}_{n_1\cdot t,j}\}$ for $0 \le i < n_1, 0 \le t < n_2$, $0 \le j < \beta$.

Output: $\hat{\mathtt{ct}}' \in (\prod_{j'=0}^{\ell-1}\mathbb{Z}_{q_{j'}}^N)^2$, the evaluation of $\boldsymbol{M}\times\hat{\mathtt{ct}}$.

Step 1 (Decompose): (1) iNTT: $\boldsymbol{c}_1 = \mathtt{iNTT}(\hat{\boldsymbol{c}}_1) \in \prod_{j'=0}^{\ell}\mathcal{R}_{q_{j'}}$. (2) Decompose: $\boldsymbol{d}_j = \mathsf{Decomp}(\boldsymbol{c}_1) \in \prod_{j'=0}^{\ell}\mathcal{R}_{q_{j'}} \times \prod_{i'=0}^{k-1}\mathcal{R}_{p_{i'}}$ for $0 \le j < \beta$. (3) NTT: Let $\hat{\boldsymbol{d}} = (\hat{\boldsymbol{d}}_j)_{0\le j<\beta}$ where $\hat{\boldsymbol{d}}_j = \mathtt{NTT}(\boldsymbol{d}_j) \in \prod_{j'=0}^{\ell}\mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1}\mathbb{Z}_{p_{i'}}^N$.

Step 2 (MultSum): (1) Compute $P\cdot\hat{\boldsymbol{c}}_0$: $\hat{\boldsymbol{c}}'_0 = ([P\cdot\hat{\boldsymbol{c}}_0^{(0)}]_{q_0}, \cdots, [P\cdot\hat{\boldsymbol{c}}_0^{(\ell)}]_{q_\ell}, \boldsymbol{0}, \cdots, \boldsymbol{0}) \in \prod_{j'=0}^{\ell}\mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1}\mathbb{Z}_{p_{i'}}^N$. (2) MultSum: For $0 \le i < n_1$, compute $(\hat{\boldsymbol{a}}_i, \hat{\boldsymbol{b}}_i) = \mathsf{MultSum}(\hat{\boldsymbol{d}}, \widehat{\mathtt{rtk}}_i)$, then let $(\hat{\boldsymbol{a}}_i, \hat{\boldsymbol{b}}_i) = (\hat{\boldsymbol{a}}_i, \hat{\boldsymbol{b}}_i) + (\hat{\boldsymbol{c}}'_0, \boldsymbol{0})$.

Step 3 (Permute): For $0 \le i < n_1$, $(\hat{\boldsymbol{a}}_i, \hat{\boldsymbol{b}}_i) = (\varphi_i(\hat{\boldsymbol{a}}_i), \varphi_i(\hat{\boldsymbol{b}}_i))$.

Step 4 (ScalarMult+Add): For $0 \le t < n_2$, compute $\hat{\boldsymbol{u}}_t = (\hat{\boldsymbol{u}}_{t,0}, \hat{\boldsymbol{u}}_{t,1}) = \sum_{i=0}^{n_1-1}\mathsf{PlainMult}(\hat{\mathtt{pt}}_{n_1\cdot t+i}, (\hat{\boldsymbol{a}}_i, \hat{\boldsymbol{b}}_i)) \in (\prod_{j'=0}^{\ell}\mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1}\mathbb{Z}_{p_{i'}}^N)^2$.

Step 5 (ModDown): (1) iNTT: For $0 \le t < n_2$, $\boldsymbol{u}_t = \mathtt{iNTT}(\hat{\boldsymbol{u}}_t)$. (2) ModDown: For $0 \le t < n_2$, $(\boldsymbol{u}_{t,0}, \boldsymbol{u}_{t,1}) = \mathsf{ModDown}(\boldsymbol{u}_t) \in (\prod_{j'=0}^{\ell}\mathcal{R}_{q_{j'}})^2$.

Step 6 (Decompose): (1) Decompose: For $0 \le t < n_2$, $\boldsymbol{w}_{t,j} = \mathsf{Decomp}(\boldsymbol{u}_{t,1}) \in \prod_{j'=0}^{\ell}\mathcal{R}_{q_{j'}} \times \prod_{i'=0}^{k-1}\mathcal{R}_{p_{i'}}$ where $0 \le j < \beta$. (2) NTT: For $0 \le t < n_2$, let $\hat{\boldsymbol{w}}_t = (\hat{\boldsymbol{w}}_{t,j})_{0\le j<\beta}$ where $\hat{\boldsymbol{w}}_{t,j} = \mathtt{NTT}(\boldsymbol{w}_{t,j})$.

Step 7 (MultSum): For $0 \le t < n_2$, compute $(\hat{\boldsymbol{r}}_{t,0}, \hat{\boldsymbol{r}}_{t,1}) = \mathsf{MultSum}(\hat{\boldsymbol{w}}_t, \widehat{\mathtt{rtk}}_{n_1\cdot t}) \in (\prod_{j'=0}^{\ell}\mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1}\mathbb{Z}_{p_{i'}}^N)^2$.

Step 8 (Permute+Add): $\hat{\boldsymbol{r}} = (\hat{\boldsymbol{r}}_0, \hat{\boldsymbol{r}}_1) = (\sum_{t=0}^{n_2-1}\varphi_{n_1\cdot t}(\hat{\boldsymbol{r}}_{t,0}), \sum_{t=0}^{n_2-1}\varphi_{n_1\cdot t}(\hat{\boldsymbol{r}}_{t,1})) \in (\prod_{j'=0}^{\ell}\mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1}\mathbb{Z}_{p_{i'}}^N)^2$. $\boldsymbol{r}_2 = \sum_{t=0}^{n_2-1}\varphi_{n_1\cdot t}(\boldsymbol{u}_{t,0}) \in \prod_{j'=0}^{\ell}\mathcal{R}_{q_{j'}}$.

Step 9 (ModDown): (1) iNTT: $\boldsymbol{r} = \mathtt{iNTT}(\hat{\boldsymbol{r}})$. (2) ModDown: $(\boldsymbol{r}_0, \boldsymbol{r}_1) = \mathsf{ModDown}(\boldsymbol{r})$. (3) $(\boldsymbol{r}_0, \boldsymbol{r}_1) = (\boldsymbol{r}_0, \boldsymbol{r}_1) + (\boldsymbol{r}_2, \boldsymbol{0})$.

Step 10 (Rescaling): (1) Rescale: $\boldsymbol{r}' = \mathsf{Rescale}((\boldsymbol{r}_0, \boldsymbol{r}_1)) \in (\prod_{j'=0}^{\ell-1}\mathcal{R}_{q_{j'}})^2$. (2) NTT: $\hat{\mathtt{ct}}' = \mathtt{NTT}(\boldsymbol{r}')$.

We show the algorithm above in RNS representation to facilitate the analysis of the time complexity (provided in Appendix E). We use $\widehat{\mathtt{rtk}}$ to represent the

NTT form of $\widetilde{\texttt{rtk}}$. The basic operations, including Decomp, MultSum, PlainMult, ModDown and Rescale, are listed in Appendix A.1 and A.2.

Roughly speaking, the time complexity of the double-hoisting BSGS algorithm can be divided into two parts: n ScalarMult and rotations, where rotations comprise four steps (Decompose, MultSum, ModDown and Permute). Also refer to [7], the complexity of these rotations can be described as

$$n_2 \cdot (Decompose + MultSum + ModDown + Permute)$$
$$+n_1 \cdot (MultSum + Permute) + Decompose + ModDown.$$

Thus the total complexity can be minimized when the ratio n_1/n_2 reaches to an optimal value.

Table 1: Time complexity of the double-hoisting BSGS algorithm in [7] for different r_1/r_2 and different r. The used parameters are $N = 2^{16}, \ell = 24, k = 5, \alpha = 5$. Mult represents the integer multiplication in $\mathbb{Z}_{q_{j'}}$ or $\mathbb{Z}_{p_{i'}}$, whose values are calculated according to the complexity analysis from Appendix E. "-" means the ratio is impossible since $r_1 \cdot r_2 = r$ and r_1, r_2 must be integers.

	r_1/r_2	log(#Mult)		r_1/r_2	log(#Mult)		r_1/r_2	log(#Mult)
$r = 8$	1	-	$r = 16$	1	30.703	$r = 32$	1	-
	2	30.019		2	-		2	30.816
	4	-		4	30.158		4	-
	8	**29.667**		8	-		8	**30.4018**
	16	-		16	**29.956**		16	-
	32	-		32	-		32	30.4021
$r = 64$	1	31.608	$r = 128$	1	-	$r = 256$	1	32.641
	2	-		2	31.778		2	-
	4	31.017		4	-		4	32.067
	8	-		8	**31.3511**		8	-
	16	**30.792**		16	-		16	**31.850**
	32	-		32	31.3514		32	-
	64	31.018		64	-		64	32.068

If we discard the BSGS method in linear transformations, it is equivalent to setting $n_1/n_2 = n$ with $n_1 = n$ and $n_2 = 1$. At first glance, this is a terrible idea. However, when we apply the matrix factorization approach [26,12], which is widely utilized in practice, to bootstrapping, the number of non-zero diagonals of the factorized matrices becomes a constant r instead of n (also with r_1, r_2 instead of n_1, n_2). Typically, the number of non-zero diagonals is around 16 or 32 (resp. 32 or 64) for 4-depth (resp. 3-depth) factorization in CoeffsToSlots[8] with $N = 2^{16}$. In this case, discarding BSGS means setting r_1/r_2 to 16, 32 or 64.

[8] "4-depth factorization in CoeffsToSlots" means factorizing the iDFT-related matrix into 4 sparse diagonal matrices, thus the depth of CoeffsToSlots is 4.

Moreover, Bossuat et al. show that the best ratio of the double-hoisting BSGS is usually 8 or 16 (Table 2 in [7]), our results in Table 1 also support it. If $r = 16$, discarding BSGS implies setting the ratio optimal, which has no efficiency loss (even has efficiency gain if the original BSGS doesn't use the optimal ratio). If $r = 32$, discarding BSGS has only a slight efficiency loss (see Table 1, 30.4021 vs. 30.4018), which can be compensated by the aggregated key-switching proposed in Section 4.2. For larger r, discarding BSGS will produce a worse efficiency, which is consistent with the common consensus. We point out that when $r = 64$, the efficiency loss of discarding BSGS is non-negligible; however, this can be compensated by adopting the improved algorithm in Section 4.2 (See Table 2 for details).

Overall, in the case of matrices with ≤ 32 non-zero diagonals, discarding BSGS seems to incur no (or small) efficiency loss for double-hoisting matrix-vector multiplication.

Remark 2. Note that while the above argument may be logically rigorous, its conclusion is relatively theoretical. In fact, counting the number of multiplications modulo $q_{j'}$ or $p_{i'}$ is not a reliable indicator of actual efficiency. Real-world efficiency is dependent on parameters, implementation, and hardware environment. Therefore, in practice, even when matrices have only a small number of non-zero diagonals, directly discarding BSGS may still incur a minor efficiency loss. We stress that discarding BSGS is not an optimization in itself; rather, it can serve as a trade-off tool to enable other optimization techniques (e.g., the proposed LCR).

4.2 Faster Matrix-Vector Multiplication for Sparse Diagonal Matrix

In this subsection, we continue to consider the sparse diagonal matrices (i.e., ≤ 64 non-zero diagonals). We propose a new key-switching technique that can reduce the time complexity of matrix-vector multiplications without BSGS method. We first show the new key-switching method (called aggregated key-switching), then provide the improved matrix-vector multiplication algorithm and the complexity analysis. We also compare the improved algorithm with the double-hoisting BSGS algorithm in [7].

Aggregated Key-Switching (AKS). To reduce the number of scalar multiplications and rescaling operations, we merge them into the key-switching procedure. Specifically, for the plaintext-ciphertext multiplication $m \times \mathtt{ct}$ with modulus Q_ℓ, and the key-switching from s_1 to s_2, we redefine the key-switching keys $\{\mathtt{evk}_j\}_{0 \leq j < \mathtt{dnum}}$ as

$$\mathtt{evk}_j = \left(\mathtt{evk}_j^0, \mathtt{evk}_j^1\right) = \left(\left[-a_j \cdot s_2 + \left\lfloor \frac{\left[Pms_1 \cdot \hat{D}_j \cdot D_j^*\right]_{PQ_L}}{q_\ell} \right\rceil + e_j \right]_{PQ_L}, [a_j]_{PQ_L} \right)$$

where $a_j \leftarrow U(\mathcal{R}_{PQ_L})$ and $e_j \leftarrow \chi_{\mathsf{err}}$ for $0 \le j < \mathtt{dnum}$[9]. In homomorphic evaluations, for $\mathtt{ct} = (c_0, c_1) \in \mathcal{R}^2_{Q_\ell}$, we perform the following procedure.

(1)**Key-Switching:** $(d_0, d_1) = \left\lfloor \frac{1}{P} \cdot \left(\sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot \mathtt{evk}_j \pmod{PQ_{\ell-1}} \right) \right\rceil \in \mathcal{R}^2_{Q_{\ell-1}}$.

(2)**ScalarMult+Rescale:** $c_0' = \mathsf{CKKS.Rescale}(m \cdot c_0) = \left\lfloor \frac{1}{q_\ell} \cdot m \cdot c_0 \right\rceil \in \mathcal{R}_{Q_{\ell-1}}$.

(3)**Addition:** $\mathtt{ct}' = (d_0, d_1) + (c_0', 0) \in \mathcal{R}^2_{Q_{\ell-1}}$.

Then, we have Theorem 2 stating that this procedure completes the key-switching, plaintext-ciphertext product and rescaling operations. The proof of Theorem 2 including the noise analysis is provided in Appendix C.

Theorem 2. *Let m be a plaintext, $\mathtt{ct}$ be a ciphertext encrypting $\mathtt{pt}$ with respect to secret s_1 and modulus Q_ℓ. Then the output of the above procedure is a valid encryption of $\frac{1}{q_\ell} \cdot m \cdot \mathtt{pt}$ with respect to s_2 and $Q_{\ell-1}$.*

Improved Matrix-Vector Multiplication. We now discuss our improved matrix-vector multiplication algorithm. As mentioned before, when using the matrix factorization method of [26,12] in bootstrapping, the DFT/iDFT matrices are decomposed into several sparse matrices with typically 16/32 non-zero diagonals, and discarding BSGS will bring no (or small) efficiency loss in this case. On this basis, we use the AKS technique for less multiplications.

For matrix $\boldsymbol{M} \in \mathbb{C}^{n \times n}$, according to Eq.(4), we encode its shifted diagonal vector into $m_i = \mathsf{Encode}(\rho_{-i}(\boldsymbol{u}_i), \Delta) \in \mathcal{R}$ for $0 \le i < n$[10]. Moreover, we write

$$\widetilde{\mathtt{rtk}}_{i,j} = \left(\widetilde{\mathtt{rtk}}^0_{i,j}, \widetilde{\mathtt{rtk}}^1_{i,j}\right) = \left(\left\lfloor -a_j \varphi_i^{-1}(s) + \left\lfloor \frac{\left[P m_i s \cdot \hat{D}_j \cdot D_j^*\right]_{PQ_L}}{q_\ell} \right\rceil + e_j \right\rfloor_{PQ_L}, [a_j]_{PQ_L} \right)$$

where $0 \le j < \mathtt{dnum}$ and i is the rotation position number. Then, we show our improved algorithm in RNS representation on the next page.

Complexity Analysis. As in Appendix E, we count the number of integer multiplications in $\mathbb{Z}_{q_{j'}}$ or $\mathbb{Z}_{p_{i'}}$ for time complexity.

- Step 1: This step is the same as the Step 1 of the algorithm in subsection 4.1, thus the complexity is $(\ell+1)N \log N + (\alpha + (\ell+k+1-\alpha)(\alpha+1))\beta N + (\ell+k+1-\alpha)\beta N \log N$ (see Appendix E).
- Step 2: Complexity of n $\mathsf{MultSum}$ operations over $\ell+k$ small moduli is $2(\ell+k)\beta n N$ (see Appendix D.4). $\mathsf{DropLevel}$ has no integer multiplication.

[9] See Appendix B.2 for the RNS version.

[10] For easy description, we still use n to denote the number of rotations. When comparing complexity, we set n to the number of non-zero diagonals r.

- Step 3: Complexity of n $\mathsf{PlainMult}$ operations for $\hat{\boldsymbol{c}}_0$ over $\ell+1$ small moduli is $(\ell+1)nN$ (see Appendix D.5).
- Step 4: There is no integer multiplications in $\mathsf{Permute}$ and additions.
- Step 5: (1) $\boldsymbol{u}$ has $2(\ell+k)$ polynomials, thus the iNTT complexity is $2(\ell+k)N\log N$. (2) The $\mathsf{ModDown}$ procedure from $\mathcal{D}_{\ell-1}$ to $\mathcal{B}_{\ell-1}$ has complexity of $2(k\ell+2\ell+k)N$ (similar to Appendix D.3).
- Step 6: (1) The iNTT complexity is $(\ell+1)N\log N$ since $\boldsymbol{r}$ has $\ell+1$ polynomials. (2) The complexity of rescaling on $\boldsymbol{r}$ is ℓN (see Appendix D.6).
- Step 7: $\boldsymbol{u}'$ has 2ℓ polynomials, thus the NTT complexity is $2\ell N\log N$.

Our improved (sparse) matrix×vector algorithm with AKS.

Input: Ciphertext $\hat{\mathtt{ct}} = (\hat{\boldsymbol{c}}_0, \hat{\boldsymbol{c}}_1) \in (\prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N)^2$, plaintexts $\{\hat{m}_i\}$ and the rotation keys $\{\widehat{\mathtt{rtk}}_{i,j}\}$ for $0 \le i < n$, $0 \le j < \beta$.

Output: $\hat{\mathtt{ct}}' \in (\prod_{j'=0}^{\ell-1} \mathbb{Z}_{q_{j'}}^N)^2$, the evaluation of $\boldsymbol{M} \times \hat{\mathtt{ct}}$.

Step 1 (Decompose): (1) iNTT: $\boldsymbol{c}_1 = \mathtt{iNTT}(\hat{\boldsymbol{c}}_1)$. (2) Decompose: $\boldsymbol{d}_j = \mathsf{Decomp}(\boldsymbol{c}_1) \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}} \times \prod_{i'=0}^{k-1} \mathcal{R}_{p_{i'}}$ for $0 \le j < \beta$. (3) NTT: Let $\hat{\boldsymbol{d}} = (\hat{\boldsymbol{d}}_j)_{0\le j<\beta}$ where $\hat{\boldsymbol{d}}_j = \mathtt{NTT}(\boldsymbol{d}_j)$.

Step 2 (MultSum): For $0 \le i < n$, compute $(\hat{\boldsymbol{a}}_i, \hat{\boldsymbol{b}}_i) = \mathsf{MultSum}(\hat{\boldsymbol{d}}, \widehat{\mathtt{rtk}}_i) \in (\prod_{j'=0}^{\ell-1} \mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1} \mathbb{Z}_{p_{i'}}^N)^2$ (This is done with an $\mathsf{RNS.DropLevel}$).

Step 3 (ScalarMult): For $0 \le i < n$, $\hat{\boldsymbol{w}}_i = \mathsf{PlainMult}(\hat{m}_i, \hat{\boldsymbol{c}}_0) \in \prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N$.

Step 4 (Permute+Add): Compute $\hat{\boldsymbol{u}} = (\hat{\boldsymbol{u}}_0, \hat{\boldsymbol{u}}_1) = (\sum_{i=0}^{n-1} \varphi_i(\hat{\boldsymbol{a}}_i), \sum_{i=0}^{n-1} \varphi_i(\hat{\boldsymbol{b}}_i))$, $\hat{\boldsymbol{r}} = \sum_{i=0}^{n-1} \varphi_i(\hat{\boldsymbol{w}}_i)$.

Step 5 (ModDown): (1) iNTT: $\boldsymbol{u} = \mathtt{iNTT}(\hat{\boldsymbol{u}})$. (2) ModDown: $(\boldsymbol{u}'_0, \boldsymbol{u}'_1) = \mathsf{ModDown}(\boldsymbol{u}) \in (\prod_{j'=0}^{\ell-1} \mathcal{R}_{q_{j'}})^2$.

Step 6 (Rescaling): (1) iNTT: $\boldsymbol{r} = \mathtt{iNTT}(\hat{\boldsymbol{r}}) \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}$. (2) Rescale: $\boldsymbol{r}' = \mathsf{Rescale}(\boldsymbol{r}) \in \prod_{j'=0}^{\ell-1} \mathcal{R}_{q_{j'}}$.

Step 7 (NTT): $\boldsymbol{u}' = (\boldsymbol{r}', \boldsymbol{0}) + (\boldsymbol{u}'_0, \boldsymbol{u}'_1)$, then $\hat{\mathtt{ct}}' = \mathtt{NTT}(\boldsymbol{u}') \in (\prod_{j'=0}^{\ell-1} \mathbb{Z}_{q_{j'}}^N)^2$.

Complexity Comparison. Here we compare the complexity of our improved matrix×vector algorithm and the double-hoisting BSGS algorithm in [7] when focusing on the sparse diagonal matrix. We assume that the number of non-zero diagonals in matrix $\boldsymbol{M}$ is r, then for the complexity, $n = r$ and $n_1 = r_1, n_2 = r_2$ with $r_1 \cdot r_2 = r$. According to the complexity analysis in Appendix E and this subsection, we compute the value of the time complexity for various values of r when N, ℓ, k and α are fixed. The results in Table 2 show that our improved algorithm without BSGS is faster than the double-hoisting BSGS algorithm when $r \le 64$, especially achieves at most 2.497× speedup when $r = 16$. For matrices with $r \ge 128$, discarding BSGS significantly increases the time complexity that even cannot be compensated by the AKS technique.

For space complexity, our algorithm needs $r-1$ rotation keys while previous BSGS algorithm requires the number of $r_1 + r_2 - 2$, which means that the im-

proved algorithm in this section is a time-memory trade-off. In subsection 5.1, we further modify it and obtain a lossless improvement.

Table 2: Time complexity comparison of the double-hoisting BSGS algorithm from [7] and our algorithm from Section 4.2 in the case of sparse diagonal matrix. The used parameters are $N = 2^{16}, \ell = 24, k = 5, \alpha = 5$. Mult represents the integer multiplication in $\mathbb{Z}_{q_{j'}}$ or $\mathbb{Z}_{p_{i'}}$. Values are calculated from "Complexity Analysis" of Appendix E and Section 4.2. "-" means the ratio is impossible since r_1, r_2 must be integers.

	BSGS in [7]		**Ours**			**BSGS in** [7]		**Ours**	
r	r_1/r_2	log(#Mult)	log(#Mult)	Speedup	r	r_1/r_2	log(#Mult)	log(#Mult)	Speedup
16	1	30.703	29.383	2.497×	32	1	-	29.940	-
	2	-		-		2	30.816		1.835×
	4	30.158		1.711×		4	-		-
	8	-		-		8	**30.4018**		1.377×
	16	**29.956**		1.488×		16	-		-
	32	-		-		32	30.4021		1.378×
64	1	31.608	30.655	1.936×	128	1	-	31.488	-
	2	-		-		2	31.778		1.223×
	4	31.017		1.285×		4	-		-
	8	-		-		8	**31.3511**		0.909×
	16	**30.792**		1.100×		16	-		-
	32	-		-		32	31.3514		0.910×
	64	31.018		1.286×		64	-		-

5 Combining Them in Bootstrapping

Now, we recall the level-conserving rescaling in Section 3. In CKKS bootstrapping, the DFT/iDFT matrix is factorized into several sparse diagonal matrices by using the matrix factorization approach [26,12] (assuming that the number of non-zero diagonals ≤ 64). It is easy to see that, LCR can save the modulus in the first matrix-vector multiplication of CoeffsToSlots, and the idea of AKS can reduce the time complexity of all the matrix-vector multiplications but requiring more rotation keys. Therefore, for the first matrix-vector multiplication of CoeffsToSlots, one can combine the two ideas since BSGS is discarded; for other matrix-vector multiplications (including those in SlotsToCoeffs), one can use the improved algorithm with AKS (or not if the memory space is insufficient).

5.1 The Combined Algorithm for the First Matrix-Vector Multiplication of CoeffsToSlots

Next, we discuss the combined algorithm (LCR+AKS) for matrix-vector multiplication (detailed in Appendix F), which contains both LCR and AKS techniques. Compared to the algorithm with AKS alone, there are *three differences*.

- Level is not reduced in the aggregated key-switching procedure.
- Rescaling is replaced by the level-conserving rescaling operation.
- Use the GHS-type key-switching instead of the hybrid method.

The first two differences are easy to understand. In the third point, we adopt the GHS-type key-switching because: the hybrid key-switching involves the CRT composition $\sum_{j=0}^{\beta-1}[c_1]_{D_j}\cdot\hat{D}_j D_j^* = c_1 + k_2 Q_\ell$ where k_2 a polynomial with integer coefficients (see Eq. (6) of Appendix C). After rescaling by q_ℓ, the term $k_2 Q_\ell$ becomes $k_2 Q_{\ell-1}$ and cannot be eliminated by modulo Q_ℓ. So we cannot maintain the modulus level after key-switching. To overcome this, we utilize the GHS-type key-switching. Normally, changing hybrid key-switching to GHS-type requires increasing the auxiliary module P. However, we do not need to increase P, we use the same auxiliary modulus P as in the hybrid key-switching, so the number of the total levels of the scheme is maintained. This can be done due to the small coefficient property of the ciphertext, which leads to a small noise (see the noise analysis below).

Concretely, in the combined algorithm for matrix-vector multiplication, we write the rotation keys (with the same notations as before) as

$$\widetilde{\mathtt{rtk}}_i = \left(\widetilde{\mathtt{rtk}}_i^0, \widetilde{\mathtt{rtk}}_i^1\right) = \left(\left[-a\varphi_i^{-1}(s) + \left\lfloor \frac{[Pm_i s]_{PQ_L}}{q_L} \right\rceil + e\right]_{PQ_L}, [a]_{PQ_L}\right) \quad (5)$$

where i is the rotation position number, $a \leftarrow U(\mathcal{R}_{PQ_L})$ and $e \leftarrow \chi_{\mathsf{err}}$. In the key-switching procedure, for $\mathtt{ct} = (c_0, c_1) \in \mathcal{R}_{Q_L}^2$ the output of ModRaise with $\|c_0\|_\infty, \|c_1\|_\infty \le q_0/2$, we perform

$$(d_0, d_1) = \left\lfloor \frac{1}{P} \cdot \left(c_1 \cdot \widetilde{\mathtt{rtk}}_i \pmod{PQ_L}\right) \right\rceil \in \mathcal{R}_{Q_L}^2.$$

In the noise analysis, one can prove that

$$\frac{1}{P}\cdot c_1 \cdot \left\lfloor \frac{[Pm_i s]_{PQ_L}}{q_L} \right\rceil = \frac{c_1}{P}\cdot\left(\frac{Pm_i s}{q_L} + r\right) = \frac{m_i \cdot c_1 s}{q_L} + \frac{c_1 r}{P}$$

since $\|Pm_i s\|_\infty \ll PQ_L/2$, where r is the rounding term with $\|r\|_\infty \le 1/2$. Let $c_0 + c_1 s = q_0 I + \mathtt{pt} + e'$. Then, following the derivation similar to the end of Appendix C, we obtain the final error formed as $\frac{m_i e'}{q_L} + \frac{c_1 r + c_1 e}{P} + e_r$, where e_r is the small noise generated by rounding operations. The c_1-related noise satisfies $\left\|\frac{c_1 r + c_1 e}{P}\right\|_\infty \le \frac{q_0 N}{4P} + \frac{q_0 N \cdot \|e\|_\infty}{2P} \ll 1$ since $q_0 \ll P$.

Two other advantages of GHS-type key-switching are: (1) it lowers the time complexity of Decomp and MultSum steps by a factor of about dnum because the

Table 3: Parameter sets used in our experiments. "Left" represents the remaining moduli after bootstrapping. $(\cdot)$ in CtS means the saved moduli by LCR. For all sets, security $\lambda \approx 128$ bits, the number of non-zero diagonals ≤ 64, $h = N/2$ and $\tilde{h} = 32$ (density of the ephemeral secret).

	Scheme Parameters							**Bootstrapping Parameters**			
Set	$\log(PQ)$	N	Δ	L	α	dnum	$\log(p_{i'})$	$\log(q_{j'})$ Left	StC	Sine	CtS
I	1767	2^{16}	2^{40}	28	6	5	$6 \cdot 61$	$60 + 13 \cdot 40$	$3 \cdot 39$	$8 \cdot 60$	$4 \cdot 56$
I′	**1751**		2^{40}	28	6	5	$6 \cdot 61$	$60 + \mathbf{14 \cdot 40}$	$3 \cdot 39$	$8 \cdot 60$	$(\mathbf{56}) + \mathbf{3 \cdot 56}$
II	1788	2^{16}	2^{45}	27	5	6	$5 \cdot 61$	$60 + 9 \cdot 45$	$3 \cdot 42$	$11 \cdot 60$	$4 \cdot 58$
II′	**1775**		2^{45}	27	5	6	$5 \cdot 61$	$60 + \mathbf{10 \cdot 45}$	$3 \cdot 42$	$11 \cdot 60$	$(\mathbf{58}) + \mathbf{3 \cdot 58}$
III	1793	2^{16}	2^{30}	25	5	6	$5 \cdot 61$	$55 + 11.5 \cdot 60$	$1.5 \cdot 60$	$8 \cdot 55$	$4 \cdot 53$
III′	1799		2^{30}	25	5	6	$5 \cdot 61$	$55 + \mathbf{12.5 \cdot 60}$	$1.5 \cdot 60$	$8 \cdot 55$	$(\mathbf{53}) + \mathbf{3 \cdot 53}$

ciphertext c_1 no longer needs to be decomposed; (2) it reduces the size of each `rtk` by a factor of `dnum`.

As results, compared with the algorithms in Section 4.1 (BSGS) and 4.2 (AKS), the combined algorithm (LCR+AKS) not only saves one level of moduli, but also further improves efficiency and reduces the total size of the rotation keys (this is shown in subsection 6.2). This makes the combined algorithm a lossless improvement. For completeness, we present the detailed operations and complexity analysis of this combined algorithm in Appendix F.

6 Experiments

We conducted experiments on the proof-of-concept implementation of the proposed techniques. Our code is developed upon the Lattigo library [34][11]. All experiments in our paper were performed with a single thread on a machine equipped with Intel(R) Core(TM) i7-9700 3.00GHz CPU and 256GB memory, running on Windows 10 Pro Version 2H22. The environment information includes Go version 1.22.1 and GOARCH amd64. We benchmark against [8] as it included the state-of-the-art linear transformation [7] and introduced the sparse-secret encapsulation technology that is mandatory in conventional dense key bootstrapping. To demonstrate the improvements, we implemented our methods based on [8]. The details of applying our methods to the sparse-secret encapsulation are presented in Section 6.3. For a relatively fair comparison, we utilized Lattigo to re-run the experiments of prior work [8] on our machine instead of using their reported values directly.

[11] https://github.com/Fainabi/Lattigo-LCR-AKS

6.1 Parameter Sets and Bootstrapping Metrics

Table 3 describes the parameters used in our experiments. I, II, III are from [8] and I′, II′, III′ are the corresponding modified parameter sets designed for our LCR method. Unlike the related work [33] that requires carefully constructing their parameters, for an existing parameter set, we directly allocate the saved moduli to the "Left" part for a stronger remaining homomorphic capacity. Moreover, our bootstrapping remains the same precision and failure probability as before since the proposed techniques have no impact on these metrics. For our modified parameter sets I′ and II′, the maximum of modulus (PQ) is smaller than previous, which might yield some benefits on security.

We also use the bootstrapping utility metric *bootstrapping throughput* in our experiments (introduced in subsection 5.3 of [12] and defined in subsection 7.1 of [7]). It is formulated as

$$throughput = \frac{n \cdot \log(\epsilon^{-1}) \cdot \log(Q_{\text{Left}})}{time},$$

where $n = N/2$ is the number of slots, $\log(\epsilon^{-1})$ is the output precision, $\log(Q_{\text{Left}})$ represents the remaining homomorphic capacity and time is the CPU time of bootstrapping.

6.2 Results

Now we discuss how to use the proposed techniques to achieve *higher throughput* bootstrapping with *reduced* `rtk` *size*. As illustrated in Fig. 1, we employ the *lossless strategy* based on the sparse-secret encapsulation bootstrapping in [8], i.e., use the combined algorithm (LCR+AKS) for the first matrix-vector multiplication in CoeffsToSlots and use the previous BSGS algorithm (from [7]) for the other matrix-vector multiplications. Experimental results are summarized in Table 4.

In Table 4, we can see that our technology lowers the running time of ModRaise and the first matrix-vector multiplication (Mat 1) in CoeffsToSlots, and saves one level of moduli, resulting in a higher bootstrapping throughput and reduced `rtk` size in CoeffsToSlots. Specifically, our ModRaise runs $9.4\times \sim 9.8\times$ faster than [8] as the second key-switching operation in the sparse-secret encapsulation is merged into the AKS procedure (refer to Section 6.3). In the first level of CoeffsToSlots (Mat 1), our LCR + AKS method accelerates the matrix-vector multiplication by a factor of $3.0 \sim 3.5$. The running time of the other matrix-vector multiplications (Mat 2, Mat 3 and Mat 4) in CoeffsToSlots of set I′ (resp. II′ and III′) is sometimes slightly longer than that of set I (resp. II and III), this is because our bootstrapping strategy performs the same (BSGS) algorithm at a higher level due to the level-conserving rescaling technique. Consequently, our advanced bootstrapping throughput reaches $1.20\times \sim 1.35\times$ that of [8] over similar parameters. Note that the bootstrapping precision and failure probability remain identical to the previous method.

Table 4: The bootstrapping performance improvements of applying our lossless strategy (I′, II′, III′) to [8] (I, II, III). Our proposal was implemented based on [8] with sparse-secret encapsulation. Mat 1, Mat 2, Mat 3 and Mat 4 denote the four-level matrix-vector multiplications of CtS, respectively, and the numbers below them denote the number of non-zero diagonals. log(bits/s) is the logarithm value of throughput. $\log(\epsilon^{-1})$ is the output precision. `rtk` size only contains the rotation keys in CtS. We sample the complex messages from the square of $-1-i$ and $1+i$ uniformly.

Bootstrapping Performance										
Set	ModRaise time(s)	CoeffsToSlots time(s)				BTS time(s)	$\log(Q_{\text{Left}})$	$\log(\epsilon^{-1})$	log(bits/s)	`rtk` size (GB)
		Mat 1 (32)	Mat 2 (15)	Mat 3 (15)	Mat 4 (31)					
I-[8]	1.14	3.62	1.85	1.79	3.34	25.98	580	27.8	24.28	5.47
I′-our	**0.12**	**1.19**	1.88	1.81	3.29	23.20	**620**	27.8	**24.54**	**4.82**
II-[8]	1.18	3.87	1.98	1.98	3.13	27.99	465	32.9	24.09	6.19
II′-our	**0.12**	**1.10**	1.81	1.73	3.24	22.91	**510**	32.9	**24.52**	**5.25**
III-[8]	1.03	3.45	1.82	1.52	2.78	23.37	745	19.7	24.29	5.81
III′-our	**0.11**	**1.03**	1.73	1.74	2.69	20.21	**805**	19.7	**24.62**	**4.93**

For space cost, LCR + AKS requires more rotation keys than [8], but the size of each `rtk` is smaller on account of the GHS-type key-switching. In the experimental results, the total size of rotation keys in CoeffsToSlots under I′-our (resp. II′-our and III′-our) is 11.9% (resp. 15.2% and 15.1%) smaller than that under I-[8] (resp. II-[8] and III-[8]).

6.3 More Discussions

Other Bootstrapping Strategies. Assume that there are $d_{\mathtt{Cts}}$ and $d_{\mathtt{StC}}$ (sparse) matrix-vector multiplications in CoeffsToSlots and SlotsToCoeffs, respectively. The above experiments show an improvement of employing LCR+AKS in the first level of CoeffsToSlots while using the BSGS approach for the other levels. In practice, if there is sufficient storage space, one can apply the algorithm with AKS to the other $d_{\mathtt{Cts}} + d_{\mathtt{StC}} - 1$ matrices to obtain a higher throughput. This is because the AKS method is faster than the BSGS algorithm but requires more `rtk` size. The more times the AKS method is utilized, the greater the efficiency gains achieved, but also the higher the memory overhead incurred. We need to decide which matrices use the AKS method according to the storage resources. It is essentially an optimization problem.

In Appendix G, we discuss how to find an appropriate time-memory trade-off strategy based on the available storage space. We also present a bootstrapping strategy as an example (i.e., the time-memory trade-of strategy in Fig. 1), and implement it using the parameters from Table 3 and sparse-secret encapsulation from [8]. Results in Table 9 demonstrate that, the example strategy has a boot-

strapping throughput $25\% \sim 40\%$ higher than [8] while doubling the `rtk` size in CoeffsToSlots.

Compatibility of LCR+AKS with the Existing Technologies. The state-of-the-art CKKS bootstrapping includes several noticeable technologies, and different technologies focus on different procedures. Our proposals mainly focus on the ModRaise and CoeffsToSlots procedures. More specifically, we care about two requirements:

- The LCR+AKS requires that the ciphertext coefficients remain small enough.
- The LCR+AKS and AKS methods both need to merge the plaintext-ciphertext multiplication and rescaling operations into the key-switching, thus requiring to put the scalar multiplication and rescaling forward in matrix-vector multiplication.

Therefore, the technologies that focus on EvalMod [12,27,37,7,28,38,39], or focus on linear transformations but don't conflict with the two requirements [33,45,31] (including the StC-first bootstrapping [13]), are all compatible with our methods, as well as those using CKKS as a subroutine (e.g., iterative CKKS bootstrapping [3,30], CKKS-style TFHE functional bootstrapping [1,5], and so on [4,19]). The combination of these technologies and LCR + AKS is straightforward without additional modifications.

Sparse-Secret Encapsulation. The sparse-secret encapsulation technique proposed in [8] is also compatible with our methods. We have incorporated it with our methods in the above experiments. Here we discuss how this integration is implemented. In the sparse-secret encapsulation, the authors introduced two key-switching operations, one key-switching from dense key s to sparse key $\tilde{s}$ is performed before ModRaise, while the other key-switching from sparse key $\tilde{s}$ to dense key s is performed after ModRaise. The second key-switching would make the ciphertext coefficients uniformly distributed over $\mathbb{Z}_{Q_L}$, which seems to prevent the utility of the subsequent LCR. In fact, we can directly merge the second key-switching (from $\tilde{s}$ to s) with the aggregated key-switching (from s to $\varphi_i^{-1}(s)$) into one aggregated key-switching from $\tilde{s}$ to $\varphi_i^{-1}(s)$, because they are two consecutive key-switching operations in bootstrapping (no trace computation in fully packed ciphertexts). Thus, the LCR component is still applied. This merging also reduces one switching key.

EvalRound$^+$. Most recently, Sung et al. proposed EvalRound$^+$ algorithm [45] that also saves the modulus consumption in CKKS bootstrapping. We can apply the LCR+AKS to the CtS* of EvalRound$^+$ to further save one level of moduli (along with better efficiency). Moreover, applying LCR+AKS to the additional CtS in EvalRound$^+$ can reduce its efficiency loss. As an illustration, we provided the experimental results of incorporating LCR+AKS into EvalRound$^+$, as shown in Table 6.

Table 5: Parameter set "EvalRound$^+$" is from [45] and "LCR+EvalRound$^+$" is the variant.

Set	Scheme Parameters								Bootstrapping Parameters			
	$\log(PQ)$	N	h	Δ	L	α	k	$\log(p_{i'})$	$\log(q_{j'})$			
									Left	StC	Sine	CtS
[45]	1438	2^{16}	192	2^{40}	24	5	5	$5 \cdot 61$	$60 + 9 \cdot 40$	$3 \cdot 39$	$8 \cdot 60$	$4 \cdot 29$
LCR+AKS+[45]	**1409**			2^{40}	23	5	5	$5 \cdot 61$	$60 + 9 \cdot 40$	$3 \cdot 39$	$8 \cdot 60$	$(\mathbf{29}) + \mathbf{3} \cdot \mathbf{29}$

Table 6: The bootstrapping performance improvements of applying LCR and AKS to EvalRound$^+$ [45]. We use the parameters in Table 5. "CtS time" means the running time of the additional CtS in EvalRound$^+$. $\log(\epsilon^{-1})$ is the output precision. `rtk` size only contains the rotation keys in CtS and CtS*.

Set	Bootstrapping Performance					
	ModRaise time(s)	CtS* time(s)	CtS time(s)	BTS time(s)	$\log(\epsilon^{-1})$	`rtk` size (GB)
[45]	0.26	5.43	3.50	16.99	27.30	4.69
LCR+AKS+[45]	**0.09**	**4.54**	**3.24**	**15.54**	27.30	3.99

7 Conclusion and Future Work

In this paper, we improved the RNS-CKKS bootstrapping algorithm with faster speed and less modulus consumption by exploring several techniques for the homomorphic linear transformations. We first introduced the level-conserving rescaling, which can be performed in CoeffsToSlots and saves the modulus. The second proposed technique aggregated key-switching works well in the matrix-vector multiplications for sparse diagonal (#(non-zero diagonals)≤ 64) matrices with lower time complexity. Prior to this work, the matrix-vector multiplication in CKKS bootstrapping is evaluated by the double-hoisting BSGS algorithm [7]. Based on the matrix factorization approach [26,12] and the GHS-type key-switching, we discarded the BSGS method and applied the two novel techniques to bootstrapping successfully for better efficiency and less modulus consumption. The implementation results demonstrate that our method increases the remaining homomorphic capacity and improves the homomorphic linear transformations significantly in CKKS bootstrapping.

One potential future research line is to explore how to apply the proposed approaches to BGV/BFV [10,9,21] or FHEW/TFHE [20,18] schemes. Additionally, taking into account the hardware-based optimizations is beneficial for implementation and realistic applications.

Acknowledgments. We thank Dr. Ming Luo for helpful discussions. We would like to thank the anonymous reviewers for their helpful suggestions. This work is supported by the Strategic Priority Research Program of the Chinese Academy of Sciences under Grant XDB0690200, and the "Climbing Program" Special Project for Basic and Frontier

Research of the Institute of Information Engineering, Chinese Academy of Sciences (No. 2024000051).

References

1. Alexandru, A., Kim, A., Polyakov, Y.: General functional bootstrapping using CKKS. In: Tauman Kalai, Y., Kamara, S.F. (eds.) Advances in Cryptology – CRYPTO 2025. pp. 304–337. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-032-01881-6_10
2. Badawi, A.A., Alexandru, A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., Kim, A., Lee, Y., Liu, Z., Micciancio, D., Pascoe, C., Polyakov, Y., Quah, I., R.V., S., Rohloff, K., Saylor, J., Suponitsky, D., Triplett, M., Vaikuntanathan, V., Zucca, V.: OpenFHE: Open-source fully homomorphic encryption library. Cryptology ePrint Archive, Paper 2022/915 (2022), https://eprint.iacr.org/2022/915
3. Bae, Y., Cheon, J.H., Cho, W., Kim, J., Kim, T.: META-BTS: Bootstrapping precision beyond the limit. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022. pp. 223–234. ACM Press (Nov 2022). https://doi.org/10.1145/3548606.3560696
4. Bae, Y., Cheon, J.H., Kim, J., Stehlé, D.: Bootstrapping bits with CKKS. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024. pp. 94–123 (2024). https://doi.org/10.1007/978-3-031-58723-8_4
5. Bae, Y., Kim, J., Stehlé, D., Suvanto, E.: Bootstrapping small integers with CKKS. In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology – ASIACRYPT 2024. pp. 330–360 (2025). https://doi.org/10.1007/978-981-96-0875-1_11
6. Bajard, J.C., Eynard, J., Hasan, M.A., Zucca, V.: A full RNS variant of FV like somewhat homomorphic encryption schemes. In: Avanzi, R., Heys, H. (eds.) Selected Areas in Cryptography – SAC 2016. pp. 423–442. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-69453-5_23
7. Bossuat, J.P., Mouchet, C., Troncoso-Pastoriza, J., Hubaux, J.P.: Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021. pp. 587–617. Springer International Publishing (2021). https://doi.org/10.1007/978-3-030-77870-5_21
8. Bossuat, J.P., Troncoso-Pastoriza, J.R., Hubaux, J.P.: Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In: Ateniese, G., Venturi, D. (eds.) ACNS 22. LNCS, vol. 13269, pp. 521–541. Springer, Heidelberg (Jun 2022). https://doi.org/10.1007/978-3-031-09234-3_26
9. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) Advances in Cryptology – CRYPTO 2012. pp. 868–886 (2012). https://doi.org/10.1007/978-3-642-32009-5_50
10. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. In: Goldwasser, S. (ed.) ITCS 2012. pp. 309–325. ACM (Jan 2012). https://doi.org/10.1145/2090236.2090262
11. Brakerski, Z., Vaikuntanathan, V.: Efficient fully homomorphic encryption from (standard) LWE. In: Ostrovsky, R. (ed.) 52nd FOCS. pp. 97–106. IEEE Computer Society Press (Oct 2011). https://doi.org/10.1109/FOCS.2011.12

12. Chen, H., Chillotti, I., Song, Y.: Improved bootstrapping for approximate homomorphic encryption. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part II. LNCS, vol. 11477, pp. 34–54. Springer, Heidelberg (May 2019). https://doi.org/10.1007/978-3-030-17656-3_2
13. Chen, H., Han, K.: Homomorphic lower digits removal and improved FHE bootstrapping. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018. pp. 315–337 (2018). https://doi.org/10.1007/978-3-319-78381-9_12
14. Cheon, J.H., Cho, W., Kim, J., Stehlé, D.: Homomorphic multiple precision multiplication for CKKS and reduced modulus consumption. In: ACM CCS 2023. p. 696–710 (2023). https://doi.org/10.1145/3576915.3623086
15. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: Bootstrapping for approximate homomorphic encryption. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018. pp. 360–384 (2018). https://doi.org/10.1007/978-3-319-78381-9_14
16. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: A full RNS variant of approximate homomorphic encryption. In: Cid, C., Jacobson Jr., M.J. (eds.) Selected Areas in Cryptography – SAC 2018. pp. 347–368. Springer International Publishing, Cham (2019). https://doi.org/10.1007/978-3-030-10970-7_16
17. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017, Part I. LNCS, vol. 10624, pp. 409–437. Springer, Heidelberg (Dec 2017). https://doi.org/10.1007/978-3-319-70694-8_15
18. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption over the torus. Journal of Cryptology **33**(1), 34–91 (Jan 2020). https://doi.org/10.1007/s00145-019-09319-x
19. Drucker, N., Moshkowich, G., Pelleg, T., Shaul, H.: BLEACH: Cleaning errors in discrete computations over CKKS. Journal of Cryptology **37** (2023). https://doi.org/10.1007/s00145-023-09483-1
20. Ducas, L., Micciancio, D.: FHEW: Bootstrapping homomorphic encryption in less than a second. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015, Part I. LNCS, vol. 9056, pp. 617–640. Springer, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46800-5_24
21. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144 (2012), https://eprint.iacr.org/2012/144
22. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Mitzenmacher, M. (ed.) 41st ACM STOC. pp. 169–178. ACM Press (May / Jun 2009). https://doi.org/10.1145/1536414.1536440
23. Gentry, C., Halevi, S., Smart, N.P.: Homomorphic evaluation of the AES circuit. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 850–867. Springer, Heidelberg (Aug 2012). https://doi.org/10.1007/978-3-642-32009-5_49
24. Halevi, S., Polyakov, Y., Shoup, V.: An improved RNS variant of the BFV homomorphic encryption scheme. In: Matsui, M. (ed.) CT-RSA 2019. LNCS, vol. 11405, pp. 83–105. Springer, Heidelberg (Mar 2019). https://doi.org/10.1007/978-3-030-12612-4_5
25. Halevi, S., Shoup, V.: Faster homomorphic linear transformations in HElib. In: Shacham, H., Boldyreva, A. (eds.) CRYPTO 2018, Part I. LNCS, vol. 10991, pp. 93–120. Springer, Heidelberg (Aug 2018). https://doi.org/10.1007/978-3-319-96884-1_4

26. Han, K., Hhan, M., Cheon, J.H.: Improved homomorphic discrete fourier transforms and FHE bootstrapping. IEEE Access **7**, 57361–57370 (2019). https://doi.org/10.1109/ACCESS.2019.2913850
27. Han, K., Ki, D.: Better bootstrapping for approximate homomorphic encryption. In: Jarecki, S. (ed.) CT-RSA 2020. LNCS, vol. 12006, pp. 364–390. Springer, Heidelberg (Feb 2020). https://doi.org/10.1007/978-3-030-40186-3_16
28. Jutla, C.S., Manohar, N.: Sine series approximation of the mod function for bootstrapping of approximate HE. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022. pp. 491–520 (2022). https://doi.org/10.1007/978-3-031-06944-4_17
29. Kim, A., Polyakov, Y., Zucca, V.: Revisiting homomorphic encryption schemes for finite fields. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part III. LNCS, vol. 13092, pp. 608–639. Springer, Heidelberg (Dec 2021). https://doi.org/10.1007/978-3-030-92078-4_21
30. Kim, J., Seo, J., Song, Y.: Simpler and faster BFV bootstrapping for arbitrary plaintext modulus from CKKS. In: ACM CCS 2024. p. 2535–2546 (2024). https://doi.org/10.1145/3658644.3670302
31. Kim, J., Cheon, J.H., Yeo, Y.: Overmodraise: Reducing modulus consumption of CKKS bootstrapping. IACR Communications in Cryptology **2**(3) (2025). https://doi.org/10.62056/a3n5qjp10
32. Kim, M., Lee, D., Seo, J., Song, Y.: Accelerating HE operations from key decomposition technique. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part IV. LNCS, vol. 14084, pp. 70–92. Springer, Heidelberg (Aug 2023). https://doi.org/10.1007/978-3-031-38551-3_3
33. Kim, S., Park, M., Kim, J., Kim, T., Min, C.: EvalRound algorithm in CKKS bootstrapping. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part II. LNCS, vol. 13792, pp. 161–187. Springer, Heidelberg (Dec 2022). https://doi.org/10.1007/978-3-031-22966-4_6
34. Lattigo v6. Online: https://github.com/tuneinsight/lattigo (Aug 2024), ePFL-LDS, Tune Insight SA
35. Lee, E., Lee, J.W., Lee, J., Kim, Y.S., Kim, Y., No, J.S., Choi, W.: Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In: Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., Sabato, S. (eds.) Proceedings of the 39th International Conference on Machine Learning. vol. 162, pp. 12403–12422 (2022), https://proceedings.mlr.press/v162/lee22e.html
36. Lee, J.W., Kang, H., Lee, Y., Choi, W., Eom, J., Deryabin, M., Lee, E., Lee, J., Yoo, D., Kim, Y.S., No, J.S.: Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. IEEE Access **10**, 30039–30054 (2022). https://doi.org/10.1109/ACCESS.2022.3159694
37. Lee, J.W., Lee, E., Lee, Y., Kim, Y.S., No, J.S.: High-precision bootstrapping of RNS-CKKS homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021. pp. 618–647 (2021). https://doi.org/10.1007/978-3-030-77870-5_22
38. Lee, Y., Lee, J.W., Kim, Y.S., Kim, Y., No, J.S., Kang, H.: High-precision bootstrapping for approximate homomorphic encryption by error variance minimization. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022. pp. 551–580 (2022). https://doi.org/10.1007/978-3-031-06944-4_19

39. Li, H., Mo, W., Shen, C., Pang, L.: EvalComp: Bootstrapping based on homomorphic comparison function for CKKS. IEEE Transactions on Information Forensics and Security **20**, 1349–1361 (2025). https://doi.org/10.1109/TIFS.2024.3516553
40. Lou, Q., Jiang, L.: HEMET: A homomorphic-encryption-friendly privacy-preserving mobile neural network architecture. In: Meila, M., Zhang, T. (eds.) Proceedings of the 38th International Conference on Machine Learning. vol. 139, pp. 7102–7110 (2021), https://proceedings.mlr.press/v139/lou21a.html
41. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Gilbert, H. (ed.) EUROCRYPT 2010. LNCS, vol. 6110, pp. 1–23. Springer, Heidelberg (May / Jun 2010). https://doi.org/10.1007/978-3-642-13190-5_1
42. Ma, S., Huang, T., Wang, A., Wang, X.: Practical dense-key bootstrapping with subring secret encapsulation. In: Hanaoka, G., Yang, B.Y. (eds.) Advances in Cryptology – ASIACRYPT 2025. pp. 101–130. Springer Nature Singapore, Singapore (2026). https://doi.org/10.1007/978-981-95-5122-4_4
43. Min, S., Lee, J.W., Song, Y.: Enhanced CKKS bootstrapping with generalized polynomial composites approximation. In: Proceedings of the 20th ACM Asia Conference on Computer and Communications Security. p. 1–12. ASIA CCS'25, Association for Computing Machinery, New York, NY, USA (2025). https://doi.org/10.1145/3708821.3710824
44. Microsoft SEAL (release 4.1). https://github.com/Microsoft/SEAL (Jan 2023), microsoft Research, Redmond, WA.
45. Sung, H., Seo, S., Kim, T., Min, C.: EvalRound+ bootstrapping and its rigorous analysis for CKKS scheme. IEEE Access **13**, 140847–140866 (2025). https://doi.org/10.1109/ACCESS.2025.3595177

Supplementary Material

A Basic Operations for CKKS in RNS Representation

In RNS representation, all the polynomials are presented in either coefficient form or NTT form over small moduli, thus the homomorphic operations would be in a different format from those in subsection 2.1. We review these operations in RNS representation here for better understanding. Note that all the notations are consistent with those in the bodytext.

Before describing the RNS-CKKS scheme, we first show the fast basis conversion (from [6,24]) and modulus switching algorithms (from [16]). Let $\mathcal{B} = \{q_0, q_1, \cdots, q_L\}$, $\mathcal{C} = \{p_0, p_1, \cdots, p_{k-1}\}$ and $\mathcal{D} = \mathcal{B} \cup \mathcal{C}$[12]. For $a \in \mathbb{Z}_Q$, the RNS representation of a in coefficient form is $[a]_{\mathcal{B}} = (a_0, \cdots, a_L) \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_L}$, one can convert it into the RNS representation over basis $\mathcal{C}$ by computing

$$\mathsf{Conv}_{\mathcal{B}\to\mathcal{C}}([a]_{\mathcal{B}}) = \left(\sum_{j=0}^{L} \left[a_j \cdot q_j^*\right]_{q_j} \cdot \hat{q}_j - v \cdot Q_L \pmod{p_i} \right)_{0 \le i < k}$$

where $\hat{q}_j = \prod_{j' \neq j} q_{j'}$, $q_j^* = \left[\hat{q}_j^{-1}\right]_{q_j}$ and $v = \left\lfloor \sum_{j=0}^{L} \frac{\left[a_j \cdot q_j^*\right]_{q_j}}{q_j} \right\rceil$.

Using the fast basis conversion on polynomials coefficient-wise, we can construct ModUp and $\mathsf{ModDown}$ algorithms, which is employed by the hybrid key-switching in RNS representation.

- $\mathsf{ModUp}_{\mathcal{B}\to\mathcal{D}}([a]_{\mathcal{B}})$: For $a \in \mathcal{R}_Q$, this procedure converts $[a]_{\mathcal{B}}$ over basis $\mathcal{B}$, to $[a]_{\mathcal{D}}$ over basis $\mathcal{D}$.

$$\begin{aligned} \mathsf{ModUp}_{\mathcal{B}\to\mathcal{D}}(\cdot) : \prod_{j=0}^{L} \mathcal{R}_{q_j} &\to \prod_{j=0}^{L} \mathcal{R}_{q_j} \times \prod_{i=0}^{k-1} \mathcal{R}_{p_i} \\ : [a]_{\mathcal{B}} &\mapsto ([a]_{\mathcal{B}}, \mathsf{Conv}_{\mathcal{B}\to\mathcal{C}}([a]_{\mathcal{B}})) \end{aligned}$$

- $\mathsf{ModDown}_{\mathcal{D}\to\mathcal{B}}([a]_{\mathcal{D}})$: For $a \in \mathcal{R}_{PQ}$ presented by $[a]_{\mathcal{D}}$, this procedure returns $\left[\left\lfloor \frac{a}{P} \right\rceil\right]_{\mathcal{B}}$. We know that $[a]_{\mathcal{D}} = ([a]_{\mathcal{B}}, [a]_{\mathcal{C}})$. Firstly, convert $[a]_{\mathcal{C}}$ to $[a \bmod P]_{\mathcal{B}}$ over basis $\mathcal{B}$, denoted by $[b]_{\mathcal{B}} = \mathsf{Conv}_{\mathcal{C}\to\mathcal{B}}([a]_{\mathcal{C}})$. Then, return $(a'_0, a'_1, \cdots, a'_L)$ where $a'_j = \left(\prod_{i=0}^{k-1} p_i\right)^{-1} \cdot (a_j - b_j) \pmod{q_j}$ for $0 \le j \le L$. That is

$$\begin{aligned} \mathsf{ModDown}_{\mathcal{D}\to\mathcal{B}}(\cdot) : \prod_{j=0}^{L} \mathcal{R}_{q_j} \times \prod_{i=0}^{k-1} \mathcal{R}_{p_i} &\to \prod_{j=0}^{L} \mathcal{R}_{q_j} \\ : ([a]_{\mathcal{B}}, [a]_{\mathcal{C}}) &\mapsto ([a]_{\mathcal{B}} - \mathsf{Conv}_{\mathcal{C}\to\mathcal{B}}([a]_{\mathcal{C}})) \cdot \left[P^{-1}\right]_{\mathcal{B}}. \end{aligned}$$

[12] For easy of description, we consider all the moduli of $\mathcal{B}$. In fact, the RNS basis sets are $\mathcal{B}_\ell = \{q_0, q_1, \cdots, q_\ell\}$ and $\mathcal{D}_\ell = \mathcal{B}_\ell \cup \mathcal{C}$ for $0 \le \ell \le L$, and the number of separated segments $\beta = \lceil (\ell + 1)/\alpha \rceil$.

A.1 Operations in Coefficient Form

Then, we detail the RNS version of the CKKS scheme by mainly focusing on the differences from the subsection 2.1.

- $\mathsf{RNS.Setup}(1^\lambda)$: Generate N, $\{q_0, \cdots q_L\}$, $\{p_0, \cdots p_{k-1}\}$, $Q, P, \{D_j\}_{0 \le j < \mathtt{dnum}}$, h, σ and instantiate distribution $\chi_{\mathsf{key}}, \chi_{\mathsf{err}}$ as in $\mathsf{CKKS.Setup}(1^\lambda)$. Moreover, we let $\mathcal{B}_\ell = \{q_0, q_1, \cdots, q_\ell\}$, $\mathcal{D}_\ell = \mathcal{B}_\ell \cup \mathcal{C}$ and $\beta = \lceil (\ell+1)/\alpha \rceil$ for $0 \le \ell \le L$, $\hat{p}_{i'} = P/p_{i'}$ for $0 \le i' < k$ and $\hat{q}_{\ell,j'} = Q_\ell / q_{j'}$ for $0 \le j' \le \ell \le L$, compute the following numbers:
  - $[\hat{p}_{i'}]_{q_{j'}}$ and $\left[\hat{p}_{i'}^{-1}\right]_{p_{i'}}$ for $0 \le i' < k, 0 \le j' \le L$.
  - $\left[P^{-1}\right]_{q_{j'}} = \left(\prod_{i'=0}^{k-1} p_{i'}\right)^{-1} \pmod{q_{j'}}$ for $0 \le j' \le L$.
  - $[\hat{q}_{\ell,j'}]_{p_{i'}}$ and $\left[\hat{q}_{\ell,j'}^{-1}\right]_{q_{j'}}$ for $0 \le i' < k, 0 \le j' \le \ell \le L$.
  - $[Q_\ell]_{p_{i'}} = \left(\prod_{j'=0}^{\ell} q_{j'}\right) \pmod{p_{i'}}$ for $0 \le i' < k, 0 \le \ell \le L$.
  - $[P]_{q_{j'}} = \left(\prod_{i'=0}^{k-1} p_{i'}\right) \pmod{q_{j'}}$ for $0 \le j' \le L$.
- $\mathsf{RNS.SwitchKeyGen}(s_1, s_2)$: To generate the switching key for key-switching from s_1 to s_2, we sample $\left(a_j^{(0)}, \cdots, a_j^{(L+k)}\right) \leftarrow U\left(\prod_{j'=0}^{L} \mathcal{R}_{q_{j'}} \times \prod_{i'=0}^{k-1} \mathcal{R}_{p_{i'}}\right)$ and $e_j \leftarrow \chi_{\mathsf{err}}$ for $0 \le j < \mathtt{dnum}$. Then the switching key $\{\mathtt{evk}_j\}_{0 \le j < \mathtt{dnum}}$ is formatted as

$$\left(\mathtt{evk}_j^{(0)} = \left(b_j^{(0)}, a_j^{(0)}\right), \cdots, \mathtt{evk}_j^{(L+k)} = \left(b_j^{(L+k)}, a_j^{(L+k)}\right)\right) \in \prod_{j'=0}^{L} \mathcal{R}_{q_{j'}}^2 \times \prod_{i'=0}^{k-1} \mathcal{R}_{p_{i'}}^2$$

  where $b_j^{(j')} = -a_j^{(j')} \cdot s_2 + [P]_{q_{j'}} \cdot \left[D_j^* \cdot \hat{D}_j\right]_{q_{j'}} \cdot s_1 + e_j \pmod{q_{j'}}$ with $\hat{D}_j = Q/D_j$, $D_j^* = \left[\hat{D}_j^{-1}\right]_{D_j}$ for $0 \le j' \le L$, and $b_j^{(L+1+i')} = -a_j^{(L+1+i')} \cdot s_2 + e_j \pmod{p_{i'}}$ for $0 \le i' < k$.
- $\mathsf{RNS.KeyGen}(\cdot)$: The processes of producing $\mathtt{sk} = (1, s), \mathtt{rlk}, \mathtt{rtk}$ and $\mathtt{conjk}$ are similar to $\mathsf{CKKS.KeyGen}(\cdot)$. For $\mathtt{pk}$, we sample $(a^{(0)}, \cdots, a^{(L)}) \leftarrow U\left(\prod_{j'=0}^{L} \mathcal{R}_{q_{j'}}\right)$ and $e \leftarrow \chi_{\mathsf{err}}$, then the public key is

$$\mathtt{pk} = \left(\mathtt{pk}^{(j')} = \left(b^{(j')}, a^{(j')}\right) \in \mathcal{R}_{q_{j'}}^2\right)_{0 \le j' \le L}$$

  where $b^{(j')} = -a^{(j')} \cdot s + e \pmod{q_{j'}}$ for $0 \le j' \le L$.
- $\mathsf{RNS.Enc}_{\mathtt{pk}}(\mathtt{pt})$: For $\mathtt{pt} \in \mathcal{R}$, sample $v \leftarrow \chi_{\mathsf{enc}}$ and $e_0, e_1 \leftarrow \chi_{\mathsf{err}}$, compute $\mathtt{ct} = \left(\mathtt{ct}^{(j')}\right)_{0 \le j' \le L} \in \prod_{j'=0}^{L} \mathcal{R}_{q_{j'}}^2$, where $\mathtt{ct}^{(j')} = v \cdot \mathtt{pk}^{(j')} + (\mathtt{pt} + e_0, e_1) \pmod{q_{j'}}$ for $0 \le j' \le L$.
- $\mathsf{RNS.Dec}_{\mathtt{sk}}(\mathtt{ct})$: For $\mathtt{ct} = \left(\mathtt{ct}^{(j')}\right)_{0 \le j' \le \ell} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2$, compute $\mathtt{pt} = \left\langle \mathtt{ct}^{(0)}, \mathtt{sk} \right\rangle \pmod{q_0} \in \mathcal{R}$.

– $\mathsf{RNS.Add}(\mathtt{ct}_1, \mathtt{ct}_2)$: For $\mathtt{ct}_r = \left(\mathtt{ct}_r^{(j')}\right)_{0\le j'\le \ell} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2$ with $r = 1, 2$, compute $\mathtt{ct}_{\mathsf{Add}} = \left(\mathtt{ct}_{\mathsf{Add}}^{(j')}\right)_{0\le j'\le \ell} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2$, where $\mathtt{ct}_{\mathsf{Add}}^{(j')} = \mathtt{ct}_1^{(j')} + \mathtt{ct}_2^{(j')} \pmod{q_{j'}}$ for $0 \le j' \le \ell$.

– $\mathsf{RNS.KeySwitch}(\boldsymbol{d}, \mathtt{evk})$: For polynomials $\boldsymbol{d} = \left(d^{(j')}\right)_{0\le j'\le \ell} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}$, let S_j denote the basis set $\{q_{j'} | j\alpha \le j' < (j+1)\alpha\}$ for $0 \le j < \beta - 1$ and $\bar{S}_{\beta-1} = \{q_{j'} | (\beta-1)\alpha \le j' \le \ell\}$, we know that $\boldsymbol{d} == \left([\boldsymbol{d}]_{S_0}, \cdots, [\boldsymbol{d}]_{S_{\beta-2}}, [\boldsymbol{d}]_{\bar{S}_{\beta-1}}\right)$. Key-switching contains three steps: $\mathsf{Decomp}, \mathsf{MultSum}, \mathsf{ModDown}$.

- $\mathsf{Decomp}(\boldsymbol{d})$: Compute $\tilde{\boldsymbol{d}}_j = \mathsf{ModUp}_{S_j \to \mathcal{D}_\ell}\left([\boldsymbol{d}]_{S_j}\right)$ for $0 \le j < \beta - 1$ and $\tilde{\boldsymbol{d}}_{\beta-1} = \mathsf{ModUp}_{\bar{S}_{\beta-1} \to \mathcal{D}_\ell}\left([\boldsymbol{d}]_{\bar{S}_{\beta-1}}\right)$[13], where each $\tilde{\boldsymbol{d}}_j \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}} \times \prod_{i'=0}^{k-1} \mathcal{R}_{p_{i'}}$. Then $\tilde{\boldsymbol{d}} = \left(\tilde{\boldsymbol{d}}_j\right)_{0\le j<\beta}$.
- $\mathsf{MultSum}(\tilde{\boldsymbol{d}}, \mathtt{evk})$: Compute

$$\tilde{\mathtt{ct}} = \left(\tilde{\mathtt{ct}}^{(0)} = \left(\tilde{\mathtt{ct}}_0^{(0)}, \tilde{\mathtt{ct}}_1^{(0)}\right), \cdots, \tilde{\mathtt{ct}}^{(\ell+k)} = \left(\tilde{\mathtt{ct}}_0^{(\ell+k)}, \tilde{\mathtt{ct}}_1^{(\ell+k)}\right)\right) \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2 \times \prod_{i'=0}^{k-1} \mathcal{R}_{p_{i'}}^2$$

  where $\tilde{\mathtt{ct}}^{(j')} = \sum_{j=0}^{\beta-1} \tilde{d}_j^{(j')} \cdot \mathtt{evk}_j^{(j')} \pmod{q_{j'}}$ for $0 \le j' \le \ell$ and $\tilde{\mathtt{ct}}^{(\ell+1+i')} = \sum_{j=0}^{\beta-1} \tilde{d}_j^{(\ell+1+i')} \cdot \mathtt{evk}_j^{(L+1+i')} \pmod{p_{i'}}$ for $0 \le i' < k$.
- $\mathsf{ModDown}(\tilde{\mathtt{ct}})$: Let $\tilde{\mathtt{ct}}_0 = \left(\tilde{\mathtt{ct}}_0^{(0)}, \cdots, \tilde{\mathtt{ct}}_0^{(\ell+k)}\right), \tilde{\mathtt{ct}}_1 = \left(\tilde{\mathtt{ct}}_1^{(0)}, \cdots, \tilde{\mathtt{ct}}_1^{(\ell+k)}\right)$. Compute

$$\mathtt{ct} = \left(\mathtt{ct}^{(0)} = \left(\mathtt{ct}_0^{(0)}, \mathtt{ct}_1^{(0)}\right), \cdots, \mathtt{ct}^{(\ell)} = \left(\mathtt{ct}_0^{(\ell)}, \mathtt{ct}_1^{(\ell)}\right)\right) \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2$$

  where

$$\left(\mathtt{ct}_0^{(0)}, \cdots, \mathtt{ct}_0^{(\ell)}\right) = \mathsf{ModDown}_{\mathcal{D}_\ell \to \mathcal{B}_\ell}(\tilde{\mathtt{ct}}_0),$$
$$\left(\mathtt{ct}_1^{(0)}, \cdots, \mathtt{ct}_1^{(\ell)}\right) = \mathsf{ModDown}_{\mathcal{D}_\ell \to \mathcal{B}_\ell}(\tilde{\mathtt{ct}}_1).$$

  Finally, output $\mathtt{ct}$.

– $\mathsf{RNS.Mult}(\mathtt{ct}, \bar{\mathtt{ct}}, \mathtt{rlk})$: For $\mathtt{ct} = \left(\mathtt{ct}^{(j')} = \left(c_0^{(j')}, c_1^{(j')}\right)\right)_{0\le j'\le \ell} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2$ and $\bar{\mathtt{ct}} = \left(\bar{\mathtt{ct}}^{(j')} = \left(\bar{c}_0^{(j')}, \bar{c}_1^{(j')}\right)\right)_{0\le j'\le \ell} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2$, first compute

$$\begin{aligned} d_0^{(j')} &= c_0^{(j')} \cdot \bar{c}_0^{(j')} \pmod{q_{j'}}, \\ d_1^{(j')} &= c_0^{(j')} \cdot \bar{c}_1^{(j')} + c_1^{(j')} \cdot \bar{c}_0^{(j')} \pmod{q_{j'}}, \\ d_2^{(j')} &= c_1^{(j')} \cdot \bar{c}_1^{(j')} \pmod{q_{j'}}, \end{aligned}$$

[13] This is different from the $\mathsf{CKKS.KeySwitch}$ in section 2.1. However, they yield the same result since $[d]_{\bar{D}_{\beta-1}} \equiv [d]_{D_{\beta-1}} \pmod{\bar{D}_{\beta-1}}$ and then $[d]_{\bar{D}_{\beta-1}} \cdot \hat{D}_{\beta-1} D_{\beta-1}^* \equiv [d]_{D_{\beta-1}} \cdot \hat{D}_{\beta-1} D_{\beta-1}^* \pmod{Q_\ell}$ where $\bar{D}_{\beta-1} = \prod_{q_j \in \bar{s}_{\beta-1}} q_j$.

for $0 \le j' \le \ell$. Then, let $\mathtt{ct}' = \left(\left(d_0^{(0)}, d_1^{(0)}\right), \cdots, \left(d_0^{(\ell)}, d_1^{(\ell)}\right)\right)$ and $\boldsymbol{d} = \left(d_2^{(j')}\right)_{0\le j'\le \ell}$, output

$$\mathtt{ct}_{\mathsf{Mult}} = \mathsf{RNS.Add}(\mathtt{ct}', \mathsf{RNS.KeySwitch}(\boldsymbol{d}, \mathtt{rlk})) \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2.$$

- $\mathsf{RNS.Rescale}(\mathtt{ct})$: For $\mathtt{ct} = \left(\mathtt{ct}^{(j')} = \left(c_0^{(j')}, c_1^{(j')}\right)\right)_{0\le j'\le \ell} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2$, compute and output $\bar{\mathtt{ct}} = \left(\bar{\mathtt{ct}}^{(j')} = \left(\bar{c}_0^{(j')}, \bar{c}_1^{(j')}\right)\right)_{0\le j'\le \ell-1} \in \prod_{j'=0}^{\ell-1} \mathcal{R}_{q_{j'}}^2$ where $\bar{c}_r^{(j')} = q_\ell^{-1} \cdot \left(c_r^{(j')} - c_r^{(\ell)}\right) \pmod{q_{j'}}$ for $r = 0, 1$ and $0 \le j' < \ell$. Note that the input of $\mathsf{RNS.Rescale}$ can be $\mathtt{ct}_0 \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}$.
- $\mathsf{RNS.DropLevel}(\mathtt{ct}, k)$: For $\mathtt{ct} = \left(\mathtt{ct}^{(j')}\right)_{0\le j'\le \ell} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2$, return $\mathtt{ct}' = \left(\mathtt{ct}^{(j')}\right)_{0\le j'\le \ell-k} \in \prod_{j'=0}^{\ell-k} \mathcal{R}_{q_{j'}}^2$.

We omit the Rotate and Conjugate here because they are essentially key-switching operation.

A.2 Operations in NTT Form

We only present operations in the coefficient form above, but the NTT form is more practical in some procedures, especially involving the polynomials multiplications. Here we rewrite some operations, including the MultSum step, PlainMult operation, in NTT form, for one to understand the section 4 better. Sometimes, for ciphertext

$$\hat{\mathtt{ct}} = \left(\hat{\mathtt{ct}}^{(j')} = \left(\hat{\boldsymbol{c}}_0^{(j')}, \hat{\boldsymbol{c}}_1^{(j')}\right)\right)_{0\le j'\le \ell} \in \prod_{j'=0}^{\ell} \left(\mathbb{Z}_{q_{j'}}^N\right)^2,$$

we rewrite is as

$$\hat{\mathtt{ct}} = \left(\hat{\mathtt{ct}}_0 = \left(\hat{\boldsymbol{c}}_0^{(j')}\right)_{0\le j'\le \ell}, \hat{\mathtt{ct}}_1 = \left(\hat{\boldsymbol{c}}_1^{(j')}\right)_{0\le j'\le \ell}\right) \in \left(\prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N\right)^2$$

when needed.

The MultSum step. In NTT form, the input $\hat{\boldsymbol{d}} = \left(\hat{\boldsymbol{d}}_j\right)_{0\le j<\beta}$ where each $\hat{\boldsymbol{d}}_j = \left(\hat{\boldsymbol{d}}_j^{(0)}, \cdots, \hat{\boldsymbol{d}}_j^{(\ell+k)}\right) \in \prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1} \mathbb{Z}_{p_{i'}}^N$, and the key-switching key is

$$\left(\widehat{\mathtt{evk}}_j^0, \widehat{\mathtt{evk}}_j^1\right) \in \left(\prod_{j'=0}^{L} \mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1} \mathbb{Z}_{p_{i'}}^N\right)^2$$

with $0 \le j < \beta$.

- $\mathsf{MultSum}(\hat{\boldsymbol{d}}, \widehat{\mathtt{evk}})$: Compute

$$\hat{\mathtt{ct}} = \left(\hat{\mathtt{ct}}_0 = \left(\hat{\mathtt{ct}}_0^{(0)}, \cdots, \hat{\mathtt{ct}}_0^{(\ell+k)}\right), \hat{\mathtt{ct}}_1 = \left(\hat{\mathtt{ct}}_1^{(0)}, \cdots, \hat{\mathtt{ct}}_1^{(\ell+k)}\right)\right) \in \left(\prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1} \mathbb{Z}_{p_{i'}}^N\right)^2$$

where

$$\left(\hat{\mathtt{ct}}_0^{(j')}, \hat{\mathtt{ct}}_1^{(j')}\right) = \left(\sum_{j=0}^{\beta-1} \hat{\boldsymbol{d}}_j^{(j')} \odot \widehat{\mathtt{evk}}_j^{0,(j')}, \sum_{j=0}^{\beta-1} \hat{\boldsymbol{d}}_j^{(j')} \odot \widehat{\mathtt{evk}}_j^{1,(j')}\right) \pmod{q_{j'}}$$

for $0 \le j' \le \ell$, and

$$\left(\hat{\mathtt{ct}}_0^{(\ell+1+i')}, \hat{\mathtt{ct}}_1^{(\ell+1+i')}\right) = \left(\sum_{j=0}^{\beta-1} \hat{\boldsymbol{d}}_j^{(\ell+1+i')} \odot \widehat{\mathtt{evk}}_j^{0,(L+1+i')}, \sum_{j=0}^{\beta-1} \hat{\boldsymbol{d}}_j^{(\ell+1+i')} \odot \widehat{\mathtt{evk}}_j^{1,(L+1+i')}\right) \pmod{p_{i'}}$$

for $0 \le i' < k$.

In GHS-type key-switching, we write $\mathsf{MultSum}$ as $\mathsf{MultKey}$, which is the special form of $\mathsf{MultSum}$ operation when $\beta = 1$.

The plaintext-ciphertext Multiplication. For the input

$$\hat{\mathtt{pt}} = \left(\hat{\mathtt{pt}}^{(j')}\right)_{0 \le j' \le \ell} \in \prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N,$$

$$\hat{\mathtt{ct}} = \left(\hat{\mathtt{ct}}_0 = \left(\hat{\boldsymbol{c}}_0^{(j')}\right)_{0 \le j' \le \ell}, \hat{\mathtt{ct}}_1 = \left(\hat{\boldsymbol{c}}_1^{(j')}\right)_{0 \le j' \le \ell}\right) \in \left(\prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N\right)^2,$$

the plaintext-ciphertext multiplication algorithm is presented as follows.

- $\mathsf{PlainMult}(\hat{\mathtt{pt}}, \hat{\mathtt{ct}})$: Compute

$$\hat{\mathtt{ct}}_{\mathtt{pt}} = \left(\hat{\mathtt{ct}}_{\mathtt{pt},0} = \left(\hat{\boldsymbol{c}}_{\mathtt{pt},0}^{(j')}\right)_{0 \le j' \le \ell}, \hat{\mathtt{ct}}_{\mathtt{pt},1} = \left(\hat{\boldsymbol{c}}_{\mathtt{pt},1}^{(j')}\right)_{0 \le j' \le \ell}\right) \in \left(\prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N\right)^2$$

  where

$$\hat{\boldsymbol{c}}_{\mathtt{pt},0}^{(j')} = \hat{\mathtt{pt}}^{(j')} \odot \hat{\boldsymbol{c}}_0^{(j')} \pmod{q_{j'}}, \quad \hat{\boldsymbol{c}}_{\mathtt{pt},1}^{(j')} = \hat{\mathtt{pt}}^{(j')} \odot \hat{\boldsymbol{c}}_1^{(j')} \pmod{q_{j'}}$$

  for $0 \le j' \le \ell$.

Note that $\mathsf{PlainMult}$ can also be performed on the input $\hat{\mathtt{ct}}_{\mathtt{pt},0} \in \prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N$ (or $\hat{\mathtt{ct}}_{\mathtt{pt},1}$) instead of $\hat{\mathtt{ct}}_{\mathtt{pt}}$.

B Our Proposed Operations in RNS Representation

B.1 level-conserving Rescaling

We show the level-conserving rescaling operation in RNS as follow:

- $\mathsf{RNS.LCRescale}(\mathtt{ct})$: For $\mathtt{ct} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2$, first compute

$$\bar{\mathtt{ct}} = \mathsf{RNS.Rescale}(\mathtt{ct}) = \left(\bar{\mathtt{ct}}^{(j')} = \left(\bar{c}_0^{(j')}, \bar{c}_1^{(j')}\right)\right)_{0 \le j' \le \ell-1} \in \prod_{j'=0}^{\ell-1} \mathcal{R}_{q_{j'}}^2.$$

  Let $\bar{c}_0 = \left(\bar{c}_0^{(0)}, \cdots, \bar{c}_0^{(\ell-1)}\right)$, $\bar{c}_1 = \left(\bar{c}_1^{(1)}, \cdots, \bar{c}_1^{(\ell-1)}\right)$, and output

$$\mathtt{ct}' = \left(\mathtt{ct}'^{(0)} = \left(c'^{(0)}_0, c'^{(0)}_1\right), \cdots, \mathtt{ct}'^{(\ell)} = \left(c'^{(\ell)}_0, c'^{(\ell)}_1\right)\right) \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2$$

  where

$$\left(c'^{(0)}_0, \cdots, c'^{(\ell)}_0\right) = \mathsf{ModUp}_{\mathcal{B}_{\ell-1} \to \mathcal{B}_\ell}(\bar{c}_0),$$

$$\left(c'^{(0)}_1, \cdots, c'^{(\ell)}_0\right) = \mathsf{ModUp}_{\mathcal{B}_{\ell-1} \to \mathcal{B}_\ell}(\bar{c}_1).$$

We also note that the input of $\mathsf{RNS.LCRescale}$ can be $\mathtt{ct}_0 \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}$.

B.2 Aggregated Key-Switching Keys

Here, we point out that the aggregated key-switching procedure is almost the same as the ordinary key-switching (may including one more $\mathsf{RNS.DropLevel}$ after $\mathsf{MultSum}$), so we only show the aggregated key-switching keys in RNS representation as follows.

To generate the switching key for key-switching from s_1 to s_2, we sample $\left(a_j^{(0)}, \cdots, a_j^{(L+k)}\right) \leftarrow U\left(\prod_{j'=0}^{L} \mathcal{R}_{q_{j'}} \times \prod_{i'=0}^{k-1} \mathcal{R}_{p_{i'}}\right)$, $e_j \leftarrow \chi_{\mathsf{err}}$ for $0 \le j < \mathtt{dnum}$, then the switching keys $\{\mathtt{evk}_j\}_{0 \le j < \mathtt{dnum}}$ is

$$\left(\mathtt{evk}_j^{(0)} = \left(b_j^{(0)}, a_j^{(0)}\right), \cdots, \mathtt{evk}_j^{(L+k)} = \left(b_j^{(L+k)}, a_j^{(L+k)}\right)\right) \in \prod_{j'=0}^{L} \mathcal{R}_{q_{j'}}^2 \times \prod_{i'=0}^{k-1} \mathcal{R}_{p_{i'}}^2$$

where

$$b_j^{(j')} = -a_j^{(j')} \cdot s_2 + \left[\left\lfloor \frac{\left[Pms_1 \cdot \hat{D}_j \cdot D_j^*\right]_{PQ_L}}{q_\ell} \right\rceil\right]_{q_{j'}} + e_j \pmod{q_{j'}}$$

for $0 \le j' \le L$, and

$$b_j^{(L+1+i')} = -a_j^{(L+1+i')} \cdot s_2 + \left[\left[\frac{\left[Pms_1 \cdot \hat{D}_j \cdot D_j^* \right]_{PQ_L}}{q_\ell} \right] \right]_{p_{i'}} + e_j \pmod{p_{i'}}$$

for $0 \le i' < k$. Note that the implementation of $\left\lfloor \frac{\left[Pms_1 \cdot \hat{D}_j \cdot D_j^* \right]_{PQ_L}}{q_\ell} \right\rceil \in \mathcal{R}_{PQ_L}$ in RNS involves a rescaling-like procedure and a ModUp operation, and it is done in pre-computations.

C Proof of Theorem 2

Before proving the Theorem 2, we show a lemma that is required in the following proof.

Lemma 2. *Assume* $L+1 = \alpha \cdot \texttt{dnum}$, *let* $D_j = \prod_{i=j\alpha}^{(j+1)\alpha-1} q_i$, $\hat{D}_j = Q_L/D_j$ *and* $D_j^* = \left[\hat{D}_j^{-1}\right]_{D_j}$ *denote the inverse of* $\hat{D}_j$ *modulo* D_j *for* $0 \le j < \texttt{dnum}$. *Let* $\beta = \lceil (\ell+1)/\alpha \rceil$ *with* $0 \le \ell \le L$. *Then for* $a \in \mathbb{Z}_{Q_\ell}$, *we have*

$$a = \sum_{j=0}^{\beta-1} [a]_{D_j} \cdot \hat{D}_j \cdot D_j^* \pmod{Q_\ell}.$$

Proof. According to the Chinese Remainder Theorem, we have

$$a = \sum_{j=0}^{\texttt{dnum}-1} [a]_{D_j} \cdot \hat{D}_j \cdot D_j^* \pmod{Q_L} = \sum_{j=0}^{\texttt{dnum}-1} [a]_{D_j} \cdot \hat{D}_j \cdot D_j^* + k_1 Q_L$$

for some integer k_1. Then

$$\sum_{j=0}^{\texttt{dnum}-1} [a]_{D_j} \cdot \hat{D}_j \cdot D_j^* + k_1 Q_L = \sum_{j=0}^{\beta-1} [a]_{D_j} \cdot \hat{D}_j \cdot D_j^* + \sum_{j=\beta}^{\texttt{dnum}-1} [a]_{D_j} \cdot \hat{D}_j \cdot D_j^* + k_1 Q_L.$$

We know that $Q_\ell | \hat{D}_j$ for $\beta \le j < \texttt{dnum}$, then, modulo Q_ℓ, we have

$$\sum_{j=0}^{\beta-1} [a]_{D_j} \cdot \hat{D}_j \cdot D_j^* \pmod{Q_\ell} = \sum_{j=0}^{\texttt{dnum}-1} [a]_{D_j} \cdot \hat{D}_j \cdot D_j^* \pmod{Q_\ell} = a$$

□

The following is the proof of Theorem 2.

Proof. Before deduction, we provide some results to simplify the rounding term of the `evk`.

Firstly, for $0 \leq j < \texttt{dnum}$, we have

$$\left\lfloor \frac{\left[Pms_1 \cdot \hat{D}_j \cdot D_j^*\right]_{PQ_L}}{q_\ell} \right\rceil = \frac{Pms_1 \cdot \hat{D}_j \cdot D_j^* + k_{1,j} \cdot PQ_L}{q_\ell} + r_j$$

where $k_{1,j}$ is a polynomial with integer coefficients and r_j is a polynomial with $\|r_j\|_\infty \leq \frac{1}{2}$.

Secondly, by Lemma 2, we know

$$\sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot \hat{D}_j \cdot D_j^* = c_1 + k_2 \cdot Q_\ell$$

where k_2 is a polynomial with integer coefficients.

Then, using the two equations above, we have

$$\begin{aligned}
&\sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot \left\lfloor \frac{\left[Pms_1 \cdot \hat{D}_j \cdot D_j^*\right]_{PQ_L}}{q_\ell} \right\rceil = \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot \frac{Pms_1 \cdot \hat{D}_j \cdot D_j^* + k_{1,j} \cdot PQ_L}{q_\ell} + \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot r_j \\
&= \frac{Pms_1}{q_\ell} \cdot \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot \hat{D}_j \cdot D_j^* + \frac{PQ_L}{q_\ell} \cdot \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot k_{1,j} + \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot r_j \\
&= \frac{Pms_1}{q_\ell} \cdot (c_1 + k_2 \cdot Q_\ell) + \frac{PQ_L}{q_\ell} \cdot \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot k_{1,j} + \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot r_j \\
&= \frac{Pms_1 \cdot c_1}{q_\ell} + Pms_1 \cdot k_2 Q_{\ell-1} + \frac{PQ_L}{q_\ell} \cdot \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot k_{1,j} + \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot r_j.
\end{aligned} \tag{6}$$

Next, we follow the 3 steps in the procedure for deduction. In MultSum,

$$\begin{aligned}
&\sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot \texttt{evk}_j \pmod{PQ_{\ell-1}} \\
&= \left(-\sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot a_j s_2 + \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot \left\lfloor \frac{\left[Pms_1 \cdot \hat{D}_j \cdot D_j^*\right]_{PQ_L}}{q_\ell} \right\rceil + \sum_{j=0}^{\beta-1} [c_1]_{D_j} e_j, \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot a_j \right) \pmod{PQ_{\ell-1}} \\
&= \left(-\sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot a_j s_2 + \frac{Pms_1 \cdot c_1}{q_\ell} + \sum_{j=0}^{\beta-1} [c_1]_{D_j} (r_j + e_j), \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot a_j \right) \pmod{PQ_{\ell-1}} \\
&= \left(-\sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot a_j s_2 + \frac{Pms_1 \cdot c_1}{q_\ell} + \sum_{j=0}^{\beta-1} [c_1]_{D_j} (r_j + e_j) + k_3 PQ_{\ell-1}, \sum_{j=0}^{\beta-1} [c_1]_{D_j} \cdot a_j + k_4 PQ_{\ell-1} \right)
\end{aligned}$$

where k_3, k_4 are polynomials with integer coefficients. After ModDown, it becomes

$$(d_0, d_1) = \left(-\frac{1}{P}\sum_{j=0}^{\beta-1}[c_1]_{D_j} a_j s_2 + \frac{m s_1 \cdot c_1}{q_\ell} + \frac{1}{P}\sum_{j=0}^{\beta-1}[c_1]_{D_j}(r_j + e_j) + r'_0, \frac{1}{P}\sum_{j=0}^{\beta-1}[c_1]_{D_j} a_j + r'_1 \right) \pmod{Q_{\ell-1}}$$

where r'_0, r'_1 are polynomials with $\|r'_0\|_\infty \le \frac{1}{2}$ and $\|r'_1\|_\infty \le \frac{1}{2}$.

Then, let $c'_0 = \frac{m c_0}{q_\ell} + r'_2$ where r'_2 is a polynomial with $\|r'_2\|_\infty \le \frac{1}{2}$. It is easy to verify that $[d_0 + d_1 \cdot s_2 + c'_0]_{Q_{\ell-1}} = \frac{m \cdot \mathtt{pt}}{q_\ell} + e$ by using $c_0 + c_1 s_1 = \mathtt{pt} + e' + k' Q_\ell$ where k' is a polynomial with integer coefficients. Moreover, the error term is

$$e = \frac{m e'}{q_\ell} + \frac{1}{P} \cdot \sum_{j=0}^{\beta-1}[c_1]_{D_j}(r_j + e_j) + r'_0 + r'_1 s_2 + r'_2.$$

□

D Complexity of the Basic Operations

Here we show the complexity of several basic operations. Let $\mathcal{B}_\ell = \{q_0, q_1, \cdots, q_\ell\}$, $\mathcal{C} = \{p_0, p_1, \cdots, p_{k-1}\}$ and $\mathcal{D}_\ell = \mathcal{B}_\ell \cup \mathcal{C}$ where $0 \le \ell \le L$, we count the number of multiplications in $\mathbb{Z}_{q_{j'}}$ or $\mathbb{Z}_{p_{i'}}$ for complexity. Additions would not be considered.

D.1 Fast Basis Conversion

Let $a \in \mathbb{Z}_{Q_\ell}$ be an integer. The fast basis conversion for $[a]_{\mathcal{B}_\ell} = (a_0, \cdots, a_\ell)$ from $\mathcal{B}_\ell$ to $\mathcal{C}$ is

$$\mathsf{Conv}_{\mathcal{B}_\ell \to \mathcal{C}}([a]_{\mathcal{B}_\ell}) = \left(\sum_{j=0}^{\ell} \left[a_j \cdot q^*_{\ell,j}\right]_{q_j} \cdot \hat{q}_{\ell,j} - v \cdot [Q_\ell]_{p_i} \pmod{p_i} \right)_{0 \le i < k} \tag{7}$$

where $\hat{q}_{\ell,j} = \prod_{0 \le j \le \ell, j' \ne j} q_{j'}$, $q^*_{\ell,j} = \left[\hat{q}^{-1}_{\ell,j}\right]_{q_j}$ and $v = \left\lfloor \sum_{j=0}^{\ell} \frac{\left[a_j \cdot q^*_{\ell,j}\right]_{q_j}}{q_j} \right\rceil$.

In Eq.(7), the terms $[\hat{q}_{\ell,j}]_{p_i}$, $\left[q^*_{\ell,j}\right]_{q_j}$ and $[Q_\ell]_{p_i}$ are pre-computed. Then one need to do the following computations.

(1) Compute $x_j = a_j \cdot \left[q^*_{\ell,j}\right]_{q_j} \pmod{q_j}$ for $0 \le j \le \ell$. This takes $\ell + 1$ integer multiplications.
(2) Compute v by using the results of the step (1). There is no integer multiplications (it requires floating-point division/addition and rounding operations).
(3) For each $0 \le i < k$, compute $\sum_{j=0}^{\ell}[x_j]_{p_i} \cdot [\hat{q}_{\ell,j}]_{p_i} \pmod{p_i}$ and $v \cdot [Q_\ell]_{p_i} \pmod{p_i}$, then perform the subtraction. It takes $\ell + 1 + 1$ integer multiplications for each i, thus $k(\ell + 2)$ in total.

Overall, the complexity of the fast basis conversion for integer a from $\mathcal{B}_\ell$ to $\mathcal{C}$ is $k(\ell + 2) + \ell + 1$.

D.2 The Decomp Procedure

The ModUp Operation. As shown in Appendix A, for a polynomial $a \in \mathcal{R}_{Q_\ell}$, the operation $\mathsf{ModUp}_{\mathcal{B}_\ell \to \mathcal{D}_\ell}([a]_{\mathcal{B}_\ell})$ involves N fast basis conversions from $\mathcal{B}_\ell$ to $\mathcal{C}$ for the coefficients of a. Thus the complexity is $N(k(\ell+2)+\ell+1)$.

The Decomp Procedure. As shown in RNS.KeySwitch, for $\boldsymbol{d} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}$, the Decomp procedure contains β ModUp operations, where $\beta-1$ ModUp operations from S_j to $\mathcal{D}_\ell$ for $0 \le j < \beta-1$ and 1 ModUp operation from $\bar{S}_{\beta-1}$ to $\mathcal{D}_\ell$. Note that S_j has α small moduli, and $\bar{S}_{\beta-1}$ has $\ell-\alpha(\beta-1)+1 = \alpha-(\alpha\beta-(\ell+1))$ small moduli. For easy description, we assume that $\bar{S}_{\beta-1}$ also has α small moduli.

A ModUp operation from α moduli to $\ell+k+1$ moduli requires N fast basis conversions from α moduli to $\ell+k+1-\alpha$ moduli, thus the complexity is $N(\alpha+(\ell+k+1-\alpha)(\alpha+1))$. Therefore, the overall complexity of the Decomp procedure is $\beta N(\alpha+(\ell+k+1-\alpha)(\alpha+1))$.

D.3 The ModDown Procedure

Here we discuss the complexity of the ModDown procedure, the third step of RNS.KeySwitch as shown in Appendix A.1. As a component, we show the complexity of the operation $\mathsf{ModDown}_{\mathcal{D}_\ell \to \mathcal{B}_\ell}$ first (we call it "ModDown operation" for distinction).

The ModDown Operation. As shown in Appendix A, for a polynomial $a \in \mathcal{R}_{PQ_\ell}$, the operation $\mathsf{ModDown}_{\mathcal{D}_\ell \to \mathcal{B}_\ell}([a]_{\mathcal{D}_\ell})$ is performed as follows.

$$([a]_{\mathcal{B}_\ell}, [a]_{\mathcal{C}}) \mapsto ([a]_{\mathcal{B}_\ell} - \mathsf{Conv}_{\mathcal{C}\to\mathcal{B}_\ell}([a]_{\mathcal{C}})) \cdot \left[P^{-1}\right]_{\mathcal{B}_\ell}.$$

(1) Fast basis conversions for the coefficients of polynomial a from $\mathcal{C}$ to $\mathcal{B}_\ell$ have complexity of $N(k+(\ell+1)(k+1)) = N(k\ell+\ell+2k+1)$.
(2) After the subtractions, it involves the multiplication with $[P^{-1}]_{q_j}$ for $0 \le j \le \ell$. For each coefficient, it takes $\ell+1$ integer multiplications, thus the complexity in this step is $N(\ell+1)$.

Overall, the complexity of the ModDown operation for polynomial a from $\mathcal{D}_\ell$ to $\mathcal{B}_\ell$ is $N(k\ell+2\ell+2k+2)$.

The ModDown Procedure. For $\mathtt{ct} \in \prod_{j'=0}^{\ell} \mathcal{R}_{q_{j'}}^2 \times \prod_{i'=0}^{k-1} \mathcal{R}_{p_{i'}}^2$, the ModDown procedure contains 2 ModDown operations from $\mathcal{D}_\ell$ to $\mathcal{B}_\ell$. So the complexity is $2N(k\ell+2\ell+2k+2)$.

D.4 The MultSum Procedure

See the MultSum procedure in Appendix A.2, for $\hat{\boldsymbol{d}} = \left(\hat{\boldsymbol{d}}_j\right)_{0\le j<\beta}$ where each $\hat{\boldsymbol{d}}_j = \left(\hat{\boldsymbol{d}}_j^{(0)}, \cdots, \hat{\boldsymbol{d}}_j^{(\ell+k)}\right) \in \prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1} \mathbb{Z}_{p_{i'}}^N$, and key-switching keys

$$\left(\widehat{\mathtt{evk}}_j^0, \widehat{\mathtt{evk}}_j^1\right) \in \left(\prod_{j'=0}^{L} \mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1} \mathbb{Z}_{p_{i'}}^N\right)^2$$

with $0 \le j < \beta$, the procedure generates an NTT-formed ciphertext with $2(\ell+k+1)$ NTT-formed integer vectors. Each integer vectors comes from an accumulation of β Hadamard products, and one Hadamard product needs N integer multiplications. Thus the overall complexity of such a MultSum is $2\beta N(\ell+k+1)$. Furthermore, the complexity of MultKey with the same moduli is $2N(\ell+k+1)$ (i.e., $\beta = 1$).

D.5 The PlainMult Procedure

As shown in Appendix A.2, the output of the plaintext-ciphertext multiplication with $\hat{\mathtt{ct}} \in (\prod_{j'=0}^{\ell} \mathbb{Z}_{q_{j'}}^N)^2$ and $\hat{\mathtt{pt}}$ is a ciphertext formatted as $2(\ell+1)$ NTT-formed integer vectors, and each integer vector comes from a Hadamard product. Thus the complexity of such a PlainMult is $2N(\ell+1)$.

D.6 The Rescaling Operation

For $\mathtt{ct} \in \prod_{j=0}^{\ell} \mathcal{R}_{q_j}^2$, the RNS.Rescale($\mathtt{ct}$) operation performs subtractions and multiplications over $\mathbb{Z}_{q_j}$ for $0 \le j \le \ell$. The output ciphertext $\bar{\mathtt{ct}} \in \prod_{j=0}^{\ell-1} \mathcal{R}_{q_j}^2$ has 2ℓ polynomials, thus $2\ell N$ coefficients in total. Therefore, the complexity of the rescaling operation for $\mathtt{ct}$ from Q_ℓ to $Q_{\ell-1}$ is $2\ell N$.

D.7 The level-conserving Rescaling Operation

According to Appendix B.1, the level-conserving rescaling operation for a ciphertext with respect to Q_ℓ contains an ordinary rescaling operation (from Q_ℓ to $Q_{\ell-1}$) and a ModUp operation (from $\mathcal{B}_{\ell-1}$ to $\mathcal{B}_\ell$). The rescaling operation requires $2\ell N$ integer multiplications (see Appendix D.6).

For the ModUp operation from $\mathcal{B}_{\ell-1}$ to $\mathcal{B}_\ell$, it involves a fast basis conversion from $\mathcal{B}_{\ell-1}$ to q_ℓ. For each coefficient in $\mathbb{Z}_{Q_{\ell-1}}$, the fast basis conversion has a complexity of $\ell + 1 \cdot (\ell+1) = 2\ell+1$, thus the complexity of $2N$ coefficients is $2N(2\ell+1)$.

Therefore, the complexity of the level-conserving rescaling operation for $\mathtt{ct} \in \prod_{j=0}^{\ell} \mathcal{R}_{q_j}^2$ is $2\ell N + 2N(2\ell+1) = 6\ell N + 2N$.

E Complexity Analysis of the Algorithm in Section 4.1

Here we count the number of integer multiplications in $\mathbb{Z}_{q_{j'}}$ or $\mathbb{Z}_{p_{i'}}$ for complexity. To focus on the flow of the algorithm, we present the complexity of the basic operations (i.e., the Decomp procedure, the ModDown procedure, the MultSum procedure, the PlainMult procedure, rescaling and so on) in Appendix D. We assume that the NTT (resp. iNTT) operation for polynomials over $\mathcal{R}_{q_{j'}}$ or $\mathcal{R}_{p_{i'}}$ (resp. $\mathbb{Z}^N_{q_{j'}}$ or $\mathbb{Z}^N_{p_{i'}}$) has a complexity of $N \log N$. The multiplication of two polynomials (over $\mathbb{Z}^N_{q_{j'}}$ or $\mathbb{Z}^N_{p_{i'}}$) involves a Hadamard product, which takes N integer multiplications. The followings are the time complexity for each step of the algorithm in section 4.1. When analyzing the complexity, we consider more tricks than the algorithm in [7], such as reusing the NTT form polynomials in Step 1, although the gain is small.

- Step 1: (1) $\hat{\boldsymbol{c}}_1$ has $\ell+1$ NTT-formed polynomials, thus the iNTT operation has a complexity of $(\ell+1)N \log N$. (2) The complexity of Decomp procedure is $(\alpha + (\ell+k+1-\alpha)(\alpha+1))\beta N$ (see Appendix D.2). (3) For $0 \le j < \beta$, each $\boldsymbol{d}_j$ has $\ell+k+1$ polynomials, but there are only $\ell+k+1-\alpha$ polynomials need NTT transformation since the $\{\hat{\boldsymbol{d}}_j^{(j')}\}_{j\alpha \le j' < (j+1)\alpha}$[14] are provided by $\hat{\boldsymbol{c}}_1$. So the complexity of NTT is $(\ell+k+1-\alpha)\beta N \log N$.
- Step 2: (1) Computing $P \cdot c_0$ requires $(\ell+1)N$ integer multiplications. (2) Complexity of n_1 MultSum operations is $2(\ell+k+1)\beta n_1 N$ (see Appendix D.4).
- Step 3: There is no integer multiplications in Permute.
- Step 4: There are $n_1 n_2$ PlainMult operations over $\ell+k+1$ small moduli, thus the complexity is $2(\ell+k+1)nN$ (see Appendix D.5).
- Step 5: (1) For $0 \le t < n_2$, each $\hat{\boldsymbol{u}}_t$ has $2(\ell+k+1)$ NTT-formed polynomials, thus the complexity of iNTT is $2(\ell+k+1)n_2 N \log N$. (2) The complexity of n_2 ModDown operations is $2(k\ell+2\ell+2k+2)n_2 N$ (see Appendix D.3).
- Step 6: (1) The complexity of n_2 Decomp operations is $(\alpha+(\ell+k+1-\alpha)(\alpha+1))\beta n_2 N$ (see Appendix D.2). (2) For $0 \le t < n_2, 0 \le j < \beta$, each $\boldsymbol{w}_{t,j}$ has $(\ell+k+1)$ polynomials, thus the NTT complexity is $(\ell+k+1)\beta n_2 N \log N$.
- Step 7: The complexity of n_2 MultSum operations is $2(\ell+k+1)\beta n_2 N$ (see Appendix D.4).
- Step 8: There is no integer multiplications in Permute and additions.
- Step 9: (1) $\boldsymbol{r}$ has $2(\ell+k+1)$ polynomials, thus the iNTT complexity is $2(\ell+k+1)N \log N$. (2) The complexity of ModDown procedure is $2(k\ell+2\ell+2k+2)N$ (see Appendix D.3). (3) Ignore additions.
- Step 10: (1) The complexity of rescaling is $2\ell N$ (see Appendix D.6). (2) The ciphertext $\boldsymbol{r}'$ has 2ℓ polynomials, thus the NTT complexity is $2\ell N \log N$.

[14] We also ignore the bias between $\ell+1$ and $\alpha\beta$.

F The Matrix×Vector Algorithm Combining LCR and AKS in CoeffsToSlots

F.1 The Algorithm

Here we present the level-conserving matrix-vector multiplication algorithm combined with aggregated key-switching technique for the first matrix-vector multiplication of CoeffsToSlots. By using the matrix factorization approach [26,12], the iDFT matrix is factorized into d_{cts} sparse diagonal matrices. For easy description, we still denote the first sparse diagonal matrix as $\boldsymbol{M} \in \mathbb{C}^{n\times n}$ with shifted diagonal vectors $\boldsymbol{u}_i$ and encode it into $m_i = \mathsf{Encode}(\rho_{-i}(\boldsymbol{u}_i), \Delta) \in \mathcal{R}$ for $0 \le i < n$. In key-switching, we define the rotation keys as in Eq.(5), and use the GSH-type key-switching. The combined algorithm is shown as follows in RNS representation. The input ciphertext $\hat{\mathtt{ct}}$ is an output of ModRaise procedure. One can refer to the end of the MultSum step in Appendix A.2 for the MultKey operation.

The combined algorithm for the first matrix×vector in CoeffsToSlots.

Input: Ciphertext $\hat{\mathtt{ct}} = (\hat{\boldsymbol{c}}_0, \hat{\boldsymbol{c}}_1) \in (\prod_{j'=0}^{L} \mathbb{Z}_{q_{j'}}^N)^2$ is the output of ModRaise, plaintexts $\{\hat{m}_i\}$ and the rotation keys $\{\widehat{\mathtt{rtk}_i}\}$ for $0 \le i < n$.

Output: $\hat{\mathtt{ct}}' \in (\prod_{j'=0}^{L} \mathbb{Z}_{q_{j'}}^N)^2$, the evaluation of $\boldsymbol{M} \times \hat{\mathtt{ct}}$.

Step 1 (Decompose): (1) iNTT: $\boldsymbol{c}_1 = \mathtt{iNTT}(\hat{\boldsymbol{c}}_1) \in \prod_{j'=0}^{L} \mathcal{R}_{q_{j'}}$. (2) Decompose: $\boldsymbol{d} = \mathsf{ModUp}_{\mathcal{B}_L \to \mathcal{D}_L}(\boldsymbol{c}_1) \in \prod_{j'=0}^{L} \mathcal{R}_{q_{j'}} \times \prod_{i'=0}^{k-1} \mathcal{R}_{p_{i'}}$. (3) NTT: Compute $\hat{\boldsymbol{d}} = \mathtt{NTT}(\boldsymbol{d}) \in \prod_{j'=0}^{L} \mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1} \mathbb{Z}_{p_{i'}}^N$.

Step 2 (MultKey): For $0 \le i < n$, compute $(\hat{\boldsymbol{a}}_i, \hat{\boldsymbol{b}}_i) = \mathsf{MultKey}(\hat{\boldsymbol{d}}, \widehat{\mathtt{rtk}_i}) \in (\prod_{j'=0}^{L} \mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1} \mathbb{Z}_{p_{i'}}^N)^2$.

Step 3 (ScalarMult): For $0 \le i < n$, compute $\hat{\boldsymbol{w}}_i = \mathsf{PlainMult}(\hat{m}_i, \hat{\boldsymbol{c}}_0) \in \prod_{j'=0}^{L} \mathbb{Z}_{q_{j'}}^N$.

Step 4 (Permute+Add): Compute $\hat{\boldsymbol{r}} = \sum_{i=0}^{n-1} \varphi_i(\hat{\boldsymbol{w}}_i) \in \prod_{j'=0}^{L} \mathbb{Z}_{q_{j'}}^N$, $\hat{\boldsymbol{u}} = (\hat{\boldsymbol{u}}_0, \hat{\boldsymbol{u}}_1) = (\sum_{i=0}^{n-1} \varphi_i(\hat{\boldsymbol{a}}_i), \sum_{i=0}^{n-1} \varphi_i(\hat{\boldsymbol{b}}_i)) \in (\prod_{j'=0}^{L} \mathbb{Z}_{q_{j'}}^N \times \prod_{i'=0}^{k-1} \mathbb{Z}_{p_{i'}}^N)^2$.

Step 5 (ModDown): (1) iNTT: $\boldsymbol{u} = \mathtt{iNTT}(\hat{\boldsymbol{u}}) \in (\prod_{j'=0}^{L} \mathcal{R}_{q_{j'}} \times \prod_{i'=0}^{k-1} \mathcal{R}_{p_{i'}})^2$. (2) ModDown: $(\boldsymbol{u}'_0, \boldsymbol{u}'_1) = \mathsf{ModDown}(\boldsymbol{u}) \in (\prod_{j'=0}^{L} \mathcal{R}_{q_{j'}})^2$.

Step 6 (level-conserving Rescaling): (1) iNTT: $\boldsymbol{r} = \mathtt{iNTT}(\hat{\boldsymbol{r}}) \in \prod_{j'=0}^{L} \mathcal{R}_{q_{j'}}$. (2) Rescale: $\boldsymbol{r}' = \mathsf{LCRescale}(\boldsymbol{r}) \in \prod_{j'=0}^{L} \mathcal{R}_{q_{j'}}$.

Step 7 (NTT): $\boldsymbol{u}' = (\boldsymbol{r}', \boldsymbol{0}) + (\boldsymbol{u}'_0, \boldsymbol{u}'_1)$, then $\hat{\mathtt{ct}}' = \mathtt{NTT}(\boldsymbol{u}') \in (\prod_{j'=0}^{L} \mathbb{Z}_{q_{j'}}^N)^2$.

F.2 The Time Complexity

As in Appendix E, we count the number of integer multiplications in $\mathbb{Z}_{q_{j'}}$ or $\mathbb{Z}_{p_{i'}}$ for time complexity. When computing the complexity value of CoeffsToSlots, the parameter n should be set to the number of non-zero diagonals in factorized matrices.

- Step 1: (1) $\boldsymbol{c}_1$ has $L+1$ polynomials, thus the iNTT operation has a complexity of $(L+1)N\log N$. (2) The Decomp procedure involves a ModUp operation from $\mathcal{B}_L$ to $\mathcal{D}_L$, so the time complexity is $(kL+2k+L+1)N$ (refer to Appendix D.2). (3) $\boldsymbol{d}$ has $L+k+1$ polynomials, but there are only k polynomials need NTT transformation since the $\{\hat{\boldsymbol{d}}^{(j')}\}_{0\le j'\le L}$ are provided by $\hat{\boldsymbol{c}}_1$. So the complexity of NTT is $kN\log N$.
- Step 2: Complexity of n $\mathsf{MultKey}$ operations over $L+k+1$ small moduli is $2(L+k+1)nN$ (see Appendix D.4).
- Step 3: Complexity of n $\mathsf{PlainMult}$ operations for $\hat{\boldsymbol{c}}_0$ over $L+1$ small moduli is $(L+1)nN$ (see Appendix D.5).
- Step 4: There is no integer multiplications in $\mathsf{Permute}$ and additions.
- Step 5: (1) $\boldsymbol{u}$ has $2(L+k+1)$ polynomials, thus the iNTT complexity is $2(L+k+1)N\log N$. (2) The complexity of $\mathsf{ModDown}$ procedure is $2(kL+2L+2k+2)N$ (see Appendix D.3).
- Step 6: (1) The iNTT complexity is $(L+1)N\log N$ since $\boldsymbol{r}$ has $L+1$ polynomials. (2) The complexity of level-conserving rescaling on $\boldsymbol{r}$ is $3LN+N$ (see Appendix D.7).
- Step 7: $\boldsymbol{u}'$ has $2(L+1)$ polynomials, thus the NTT complexity is $2(L+1)N\log N$.

G Time-Memory Trade-Off Bootstrapping Strategies

G.1 Find the Best Strategy

In this section, we design an algorithm to find the best time-memory trade-off strategy for applying BSGS algorithm and the AKS algorithm to bootstrapping. Such algorithm is designed to be suitable for any hardware and software instantiation.

There are $d_{\mathsf{CtS}}+d_{\mathsf{StC}}-1$ matrices and our goal is to determine which levels of matrix-vector multiplication use the BSGS algorithm and the others use the AKS method. For simplicity, assuming that there are k factorized matrices (i.e., $k=d_{\mathsf{CtS}}+d_{\mathsf{StC}}-1$), and $t_{\mathsf{AKS},i}, u_{\mathsf{AKS},i}$ are the time and memory usage for evaluating the algorithm with aggregated key-switching at the level i for $1\le i\le k$ (similar notations are for $t_{\mathtt{BSGS},i}$ and $u_{\mathtt{BSGS},i}$). Our purpose is to minimize the efficiency and make sure that the required memory does not exceed the capacity. That throws a single objective optimization problem as follows.

$$\begin{aligned}&\min_{b_i}\sum_{i=1}^{k} b_i t_{\mathsf{AKS},i}+(1-b_i)t_{\mathtt{BSGS},i}\\ &\text{s.t.}\ \begin{cases}\sum_{i=1}^{k} b_i u_{\mathsf{AKS},i}+(1-b_i)u_{\mathtt{BSGS},i}\le u\\ b_i\in\{0,1\},\forall i=1,\cdots,k\end{cases}\end{aligned}\tag{8}$$

where u is the maximum memory for `rtk` provided by the machine.

The next steps are direct: measure $t_{\mathsf{AKS},i}, t_{\mathtt{BSGS},i}, u_{\mathsf{AKS},i}, u_{\mathtt{BSGS},i}$ on the target machine and solve the integer linear programming problem. As an example with

$k = 3$, we measure these parameters on our machine which are listed in Table 7. With available memory more than 8.49 GB, we can deploy the AKS method on all the matrices for the best performance.

Table 7: Measure the time and memory required at each level for the parameter set I from [7].

Matrix	1	2	3
#(Non-zero diagonals)	15	15	31
t_{BSGS} (s)	1.07	0.97	2.15
u_{BSGS} (GB)	0.88	0.88	1.46
t_{AKS} (s)	0.75	0.74	1.50
u_{AKS} (GB)	2.05	2.05	4.39

The optimization problem above aims to find the best strategy that presents best run time. One could consider more objectives such as memory usage and others. More generally, from the perspective of the scheme parameter setting, the constraints may also contain how the matrix are factorized, and we formalize them with an abstract optimization problem for completeness.

$$\min_{k,b_i,N_i} f_m(\boldsymbol{t}, \boldsymbol{u}), \quad m = 1, \cdots, M$$
$$\text{s.t.} \begin{cases} \sum_{i=1}^{k} b_i u_{\text{AKS},i} + (1 - b_i) u_{\text{BSGS},i} \leq u \\ g(N_1, \cdots, N_k, N) = 0, \\ b_i \in \{0, 1\}, \forall i = 1, \cdots, k \end{cases}$$

Here f_m is an objective that can be the total time or memory used and others, settled by users. $g(N_1, \cdots, N_k, N) = 0$ indicates a valid factorization for DFT/iDFT matrix in dimension N. $N_i, i = 1, \cdots, k$ denotes number of non-zero diagonals in the factorized matrix i.

G.2 An Example Bootstrapping Strategy and Its Results

Here, we present an example time-memory trade-off strategy (i.e., the time-memory trade-of strategy in Fig. 1) for bootstrapping, and implement it using both the parameters in Table 3 (dense key bootstrapping) and the sparse key bootstrapping parameters (as shown in Table 8). Sparse key bootstrapping has a lower latency than dense key bootstrapping. All experiments were implemented with sparse-secret encapsulation technique and the results are summarized in Table 9.

The example bootstrapping strategy. For the first matrix-vector multiplication in CoeffsToSlots, we use the LCR+AKS method; For other matrix-vector multiplications in CoeffsToSlots, we use the AKS method; For the matrix-vector multiplications in SlotsToCoeffs, we use the BSGS algorithm.

Table 8: Parameter sets for sparse key bootstrapping. $\tilde{h} = 32$ for all sets.

	Scheme Parameters								Bootstrapping Parameters					
Set	$\log(PQ)$	N	h	Δ	L	α	k	$\log(p_{i'})$	$\log(q_{j'})$ Left	$\log(q_{j'})$ StC	$\log(q_{j'})$ Sine	$\log(q_{j'})$ CtS	$d_{\mathtt{Cts}}$	$d_{\mathtt{StC}}$
1	1546	2^{16}	192	2^{40}	24	5	5	$5 \cdot 61$	$60 + 9 \cdot 40$	$3 \cdot 39$	$8 \cdot 60$	$4 \cdot 56$	4	3
$1'$	**1530**			2^{40}	24	5	5	$5 \cdot 61$	$60 + \mathbf{10} \cdot \mathbf{40}$	$3 \cdot 39$	$8 \cdot 60$	$(\mathbf{56}) + \mathbf{3} \cdot \mathbf{56}$	4	3
2	1547	2^{16}	192	2^{45}	23	4	4	$4 \cdot 61$	$60 + 5 \cdot 45$	$3 \cdot 42$	$11 \cdot 60$	$4 \cdot 58$	4	3
$2'$	**1534**			2^{45}	23	4	4	$4 \cdot 61$	$60 + \mathbf{6} \cdot \mathbf{45}$	$3 \cdot 42$	$11 \cdot 60$	$(\mathbf{58}) + \mathbf{3} \cdot \mathbf{58}$	4	3
3	1552	2^{16}	192	2^{30}	21	5	5	$5 \cdot 61$	$55 + 7.5 \cdot 60$	$1.5 \cdot 60$	$8 \cdot 55$	$4 \cdot 53$	4	3
$3'$	1559			2^{30}	21	5	5	$5 \cdot 61$	$55 + \mathbf{8.5} \cdot \mathbf{60}$	$1.5 \cdot 60$	$8 \cdot 55$	$(\mathbf{53}) + \mathbf{3} \cdot \mathbf{53}$	4	3

Results. Table 9 reports the experiment results. We can see that the example strategy further accelerates the bootstrapping (achieving up to 40% throughput speed-up), while doubles the `rtk` size in CoeffsToSlots. For example, compared to the parameter set 3-[8], $3'$-T requires 4.24 GB more rotation keys. Note that the 4.24 GB additional space overhead is fully acceptable in the real world. Especially for successive or iterative bootstrapping, once the rotation keys are generated in pre-computations, they can be used in all the bootstrapping operations. Moreover, our rotation keys are level-dependent because of the aggregation of the plaintexts and rescaling factors. Therefore, one can produce the `rtk` in $\mathcal{R}^2_{PQ_\ell}$ rather than $\mathcal{R}^2_{PQ_L}$ for $0 \leq \ell \leq L$ to further reduce the size of our `rtk`.

In practice, there are more optimizations for the AKS method to make it applicable. We refer to Appendix H for more details.

H Applicability of the AKS Method

AKS technology improves the efficiency of CKKS bootstrapping at the cost of increased key materials. Although AKS increases the rotation keys, it is also a significant contribution for applications that require many times of bootstrapping, such as the CKKS-style TFHE functional bootstrapping (single operation per bootstrapping) and the iterative CKKS bootstrapping. Keys can be reused in different bootstrapping procedures.

Additionally, the AKS keys can be also reused in other matrix multiplications beyond the bootstrapping , e.g., in privacy-preserving machine learning (PPML), by modifying the matrix encoding slightly (we elaborate this in Appendix H.2). This demonstrates the general applicability of the AKS method in real-world scenarios.

H.1 Other Optimizations for the AKS Method

In the main text, the AKS and LCR methods are used in combination. In fact, as an independent technology, AKS can be further optimized in the linear transformations. For example, the algorithm presented in page 21 can be further

Table 9: Bootstrapping performance of all experiments. Parameter set with "L" means using the lossless bootstrapping strategy from the main text while "T" means using the example time-memory trade-off strategy. `rtk` size only contains the rotation keys in CtS.

Bootstrapping Performance											
Set	ModRaise time(s)	CoeffsToSlots time(s)				BTS time(s)	$\log(Q_{\text{Left}})$	$\log(\epsilon^{-1})$	log(bit/s)	TP Speed-up	`rtk` size (GB)
		Mat 1 (32)	Mat 2 (15)	Mat 3 (15)	Mat 4 (31)						
Sparse Key											
1-[8]	0.86	2.77	1.42	1.37	2.49	19.57	420	27.8	24.22	–	4.69
1′-L	0.12	0.89	1.45	1.40	2.51	17.40	460	27.8	24.52	1.23	4.13
1′-T	0.12	1.10	1.12	1.09	1.86	16.45	460	27.8	24.60	1.30	9.40
2-[8]	0.82	2.87	1.47	1.42	2.62	19.71	285	32.9	23.89	–	5.25
2′-L	0.11	0.80	1.50	1.43	2.59	17.77	330	32.9	24.25	1.28	4.46
2′-T	0.10	0.86	1.18	1.18	1.97	16.79	330	32.9	24.34	1.37	10.36
3-[8]	0.70	2.61	1.39	1.15	2.05	16.54	505	19.7	24.23	–	4.22
3′-L	0.10	0.80	1.38	1.37	2.11	14.95	565	19.7	24.54	1.24	3.72
3′-T	0.09	0.93	1.13	1.11	1.68	14.29	565	19.7	24.61	1.30	8.46
Dense Key											
I-[8]	1.14	3.62	1.85	1.79	3.34	25.98	580	27.8	24.28	–	5.47
I′-L	0.12	1.19	1.88	1.81	3.29	23.20	620	27.8	24.54	1.20	4.82
I′-T	0.13	1.15	1.55	1.46	2.48	22.18	620	27.8	24.60	1.25	10.97
II-[8]	1.18	3.87	1.98	1.98	3.13	27.99	465	32.9	24.09	–	6.19
II′-L	0.12	1.10	1.81	1.73	3.24	22.91	510	32.9	24.52	1.35	5.25
II′-T	0.12	1.14	1.46	1.39	2.42	21.99	510	32.9	24.58	**1.40**	12.21
III-[8]	1.03	3.45	1.82	1.52	2.78	23.37	745	19.7	24.29	–	5.81
III′-L	0.11	1.03	1.73	1.74	2.69	20.21	805	19.7	24.62	1.26	4.93
III′-T	0.12	1.05	1.36	1.31	2.04	19.29	805	19.7	24.68	1.31	11.47

improved by merging the rescaling operation on c_0 (i.e., the Rescale on $\boldsymbol{r}$ in Step 6) into the ModDown (Step 5). Specifically, in the key-switching keys, we encrypt Pms instead of $\frac{Pms}{q_\ell}$. After step 4 and iNTT, we raise $\boldsymbol{r}$ to $\mathcal{R}_{PQ_\ell}$ via multiplying it by P. Then, we add $(P \cdot \boldsymbol{r}, \boldsymbol{0})$ and $(\boldsymbol{u}_0, \boldsymbol{u}_1)$, and perform the ModDown from PQ_ℓ to $Q_{\ell-1}$ (doing Rescale and ModDown together, like the modification in [43]).

As a novel independent technology, AKS pioneers a new technological paradigm: merging some operations into keys-witching, just like a "functional key-switching". We believe the idea of functional key-switching can be explored and optimized in the future, and will yield surprising results.

H.2 Reusing AKS Keys

In the main text, the rotation keys in AKS method cannot be directly reused in other operations beyond the bootstrapping, because the AKS keys contain the specific plaintexts related to iDFT/DFT matrices. Here we show how to eliminate the encoded plaintexts and reuse the AKS keys in PPML.

We now mainly focus on reusing the rotation keys in AKS method, instead of the LCR + AKS method. Furthermore, we consider the optimized AKS method, which includes only scalar multiplications in key-switching, and integrates the rescaling operation into the ModDown procedure (detailed in Appendix H.1). Notations are the same as those in the main text.

According to the optimization in Appendix H.1, the AKS keys are defined as

$$\widetilde{\texttt{rtk}}_{i,j} = \left(-a_j\varphi_i^{-1}(s) + Pm_i s \cdot \hat{D}_j \cdot D_j^* + e_j, a_j\right) \in \mathcal{R}^2_{PQ_L} \tag{9}$$

where m_i is the plaintext from the iDFT matrix, $0 \le i < n, 0 \le j <$ dnum. Assume that we need to perform a conventional homomorphic matrix-vector multiplication[15] in a real-world application, which for some indices $i \in \mathcal{A}$, involves the same rotations φ_i but scalar multiplications on other plaintexts m'_i derived from a user-specified matrix M ($\mathcal{A}$ denotes the set composed of all the same indices). Conventionally, the desired rotation keys are

$$\widetilde{\texttt{rtk}'}_{i,j} = \left(-a'_j\varphi_i^{-1}(s) + Ps \cdot \hat{D}_j \cdot D_j^* + e'_j, a'_j\right) \in \mathcal{R}^2_{PQ_L}. \tag{10}$$

Our goal is to complete the matrix-vector multiplication by reusing the iDFT-related rotation keys $\widetilde{\texttt{rtk}}$ instead of generating new keys, if the rotation index is the same (i.e., in $\mathcal{A}$). This is achieved by Alg. 1.

[15] Using the BSGS method and the hybrid key-switching.

Algorithm 1 Reusing AKS keys in matrix×vector algorithm.

Require: Ciphertext $\mathtt{ct}$, $n_1 n_2 = n$. $(v_j)_{0\le j<n}$ are the $j-$th diagonal vector of user-specified matrix $M \in \mathbb{C}^{n\times n}$. AKS keys $\widetilde{\mathtt{rtk}}$, $u_i = (u_{ik})_{0\le k\le n} \in \mathbb{C}^n$ is the messages in the slots of m_i, where $i \in \mathcal{A}$. Other keys $\widetilde{\mathtt{rtk}'}$ whose rotation indices not in $\mathcal{A}$.

Ensure: $\mathtt{ct}' = M \times \mathtt{ct}$

/**Pre-computations for encoding**/

1: **for** $0 \le k < n_1, 0 \le t < n_2$ **do** ▷ BSGS requires rotation indices k and $n_1 \cdot t$
2: $v'_{n_1 t+k} = \rho_{-n_1 t}(v_{n_1 t+k})$; ▷ The regular encoding in BSGS method
3: **if** $k \in \mathcal{A}$ **then** ▷ Check baby-step
4: $v'_{n_1 t+k} = v'_{n_1 t+k} \odot \rho_k(u_k^{-1})$; ▷ $u_i^{-1} = (\frac{1}{u_{i0}}, \cdots, \frac{1}{u_{i,n-1}}) \in \mathbb{C}^n$
5: **end if**
6: **if** $n_1 \cdot t \in \mathcal{A}$ **then** ▷ Check giant-step
7: $v'_{n_1 t+k} = v'_{n_1 t+k} \odot u_{n_1 t}^{-1}$;
8: **end if**
9: $m'_{n_1 t+k} = \mathsf{Encode}(v'_{n_1 t+k}, \Delta_M)$;
10: **end for**
11: Perform BSGS M×V algorithm using $m'_{n_1 t+k}$. ▷ $\widetilde{\mathtt{rtk}}$ for rotation index $i \in \mathcal{A}$

Explanation. The main idea of Alg. 1 is pre-encoding the u_i^{-1} into the plaintexts, to eliminate the m_i in $\widetilde{\mathtt{rtk}}$. Recall the procedures in BSGS matrix-vector multiplications (refer to Section 4.1), the Permute operation is performed between the baby-step key-switching and the scalar multiplication, so we need $\rho_k(u_k^{-1})$ when checking the baby-step (step 4 in Alg. 1). This algorithm only requires some additional Hadamard products in matrix encoding.